

铁路多目标识别与人工复核软件源代码

软件名称：铁路多目标识别与人工复核软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. backend/app/__init__.py
2. backend/app/api/__init__.py
3. backend/app/api/ai.py
4. backend/app/api/annotations.py
5. backend/app/api/assets.py
6. backend/app/api/auth.py
7. backend/app/api/export.py
8. backend/app/api/image_tasks.py
9. backend/app/api/images.py
10. backend/app/api/labels.py
11. backend/app/api/tasks.py
12. backend/app/api/video_captions.py
13. backend/app/api/vision.py
14. backend/app/core/__init__.py
15. backend/app/core/config.py
16. backend/app/core/dependencies.py
17. backend/app/core/security.py
18. backend/app/main.py
19. backend/app/models/__init__.py
20. backend/app/models/schemas.py
21. backend/app/repositories/__init__.py
22. backend/app/repositories/postgres_store.py
23. backend/app/services/__init__.py
24. backend/app/services/export_service.py
25. backend/app/services/inference_service.py
26. backend/app/services/storage_service.py
27. backend/app/services/task_service.py
28. backend/app/services/task_worker.py
29. backend/app/services/vision_service.py
30. backend/models/deepseek-vl2-tiny/config.json
31. backend/models/deepseek-vl2-tiny/processor_config.json
32. backend/pyproject.toml
33. backend/requirements-gpu.txt
34. backend/scripts/export_tensorrt.py
35. backend/scripts/process_video_captions.py
36. backend/scripts/update_video_caption_titles.py

backend/app/__init__.py

```
"""Railway recognition backend package."""
```

backend/app/api/__init__.py

```
"""HTTP route modules."""
```

backend/app/api/ai.py

```
from pathlib import Path

from fastapi import APIRouter, Depends, HTTPException

from app.api.auth import require_user
from app.core.dependencies import get_store, get_vision_service
from app.models.schemas import AiInferRequest, ApiResponse, AuthUser, ImageTaskResult
from app.repositories.postgres_store import PostgresStore
from app.services.vision_service import VisionService

router = APIRouter(prefix="/api/ai", tags=["image-workspace"])

@router.post("/infer", response_model=ApiResponse)
def infer_image_task(
    request: AiInferRequest,
    store: PostgresStore = Depends(get_store),
    service: VisionService = Depends(get_vision_service),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    if current_user.role == "reviewer":
        raise HTTPException(status_code=403, detail="审核员不能执行模型或修改任务结果")
    image = store.get_accessible_image_asset(request.image_id, current_user.user_id)
    if image is None:
        raise HTTPException(status_code=404, detail="当前图片不存在")

    session_id = request.session_id or image.session_id or "default"
    task = store.create_or_get_image_task(request.image_id, request.task_type, session_id, current_user.user_id)
    existing_result = store.get_image_task_result_by_task_id(task.task_id)
    if existing_result is not None and existing_result.review_status == "approved":
        raise HTTPException(status_code=409, detail="审核通过的任务已经锁定，不能重新执行模型")
    store.update_image_task_status(task.task_id, "processing")

    try:
        analysis = service.analyze_path(Path(image.file_path), request.task_type, request.model_name)
        result_json = _result_from_analysis(request.task_type, analysis)
    except Exception as exc:
        store.update_image_task_status(task.task_id, "failed")
        raise HTTPException(status_code=503, detail=f"通用视觉模型执行失败: {exc}") from exc

    description = result_json.get("description", "") if isinstance(result_json, dict) else ""
    model_id = str(analysis.get("model") or request.model_name or "unknown")
    result = ImageTaskResult(
        task_id=task.task_id,
        image_id=image.image_id,
        task_type=request.task_type,
        result_image_url="",
        result_image_path="",
        result_json=result_json,
        annotation_json=_annotation_from_result(request.task_type, result_json),
        description_text=description,
        source="ai",
        model_id=model_id,
    )
    saved, version = store.save_image_task_result(result)
```

```

return ApiResponse(
    data={
        "taskId": task.task_id,
        "imageId": image.image_id,
        "taskType": request.task_type,
        "status": "success",
        "resultId": saved.result_id,
        "versionId": version.version_id,
        "versionNo": version.version_no,
        "resultJson": result_json,
        "annotationJson": saved.annotation_json,
        "descriptionText": saved.description_text,
        "source": saved.source,
        "modelId": saved.model_id,
        "latestVersionId": version.version_id,
        "latestVersionNo": version.version_no,
    }
)

```

```

def _result_from_analysis(task_type: str, analysis: dict) -> dict:
    if task_type == "detection":
        return {"boxes": [_box_payload(item) for item in analysis.get("objects", [])]}
    if task_type == "segmentation":
        return {
            "segments": [
                {
                    "label": item.get("label", "object"),
                    "score": float(item.get("confidence", 0.0)),
                    "polygon": item.get("polygon", []),
                }
                for item in analysis.get("segments", [])
            ]
        }
    if task_type == "pose":
        poses = analysis.get("poses", [])
        return {"keypoints": _pose_keypoints(poses), "skeleton": _pose_skeleton(poses)}
    if task_type == "classification":
        return {
            "labels": [
                {"label": item.get("label", "unknown"), "score": float(item.get("confidence", 0.0))}
                for item in analysis.get("results", [])
            ]
        }
    return {"description": str(analysis.get("caption", ""))}

```

```

def _box_payload(item: dict) -> dict:
    bbox = item.get("bbox", {})
    return {
        "x": float(bbox.get("x", 0.0)),
        "y": float(bbox.get("y", 0.0)),
        "width": float(bbox.get("width", 1.0)),
        "height": float(bbox.get("height", 1.0)),
        "label": str(item.get("label", "object")),
        "score": float(item.get("confidence", 0.0)),
    }

```

```

KEYPOINT_NAMES = [
    "nose",
    "left_eye",
    "right_eye",
    "left_ear",
    "right_ear",
    "left_shoulder",
    "right_shoulder",
    "left_elbow",
    "right_elbow",
    "left_wrist",
    "right_wrist",

```

```

    "left_hip",
    "right_hip",
    "left_knee",
    "right_knee",
    "left_ankle",
    "right_ankle",
]

COCO_SKELETON = [
    ("left_shoulder", "right_shoulder"),
    ("left_shoulder", "left_elbow"),
    ("left_elbow", "left_wrist"),
    ("right_shoulder", "right_elbow"),
    ("right_elbow", "right_wrist"),
    ("left_shoulder", "left_hip"),
    ("right_shoulder", "right_hip"),
    ("left_hip", "right_hip"),
    ("left_hip", "left_knee"),
    ("left_knee", "left_ankle"),
    ("right_hip", "right_knee"),
    ("right_knee", "right_ankle"),
    ("nose", "left_eye"),
    ("nose", "right_eye"),
    ("left_eye", "left_ear"),
    ("right_eye", "right_ear"),
]

def _pose_keypoints(poses: list[dict]) -> list[dict]:
    keypoints: list[dict] = []
    for pose_index, pose in enumerate(poses):
        coordinates = pose.get("keypoints", [])
        confidences = pose.get("confidence", [])
        for point_index, coordinates_xy in enumerate(coordinates):
            if len(coordinates_xy) < 2:
                continue
            base_name = KEYPOINT_NAMES[point_index] if point_index < len(KEYPOINT_NAMES) else
f"point_{point_index}"
            keypoints.append(
                {
                    "name": f"person_{pose_index}_{base_name}",
                    "x": float(coordinates_xy[0]),
                    "y": float(coordinates_xy[1]),
                    "score": float(confidences[point_index]) if point_index < len(confidences) else 0.0,
                }
            )
    return keypoints

def _pose_skeleton(poses: list[dict]) -> list[dict]:
    skeleton: list[dict] = []
    for pose_index, pose in enumerate(poses):
        coordinates = pose.get("keypoints", [])
        confidences = pose.get("confidence", [])
        points: dict[str, dict] = {}
        for point_index, coordinates_xy in enumerate(coordinates):
            if len(coordinates_xy) < 2 or point_index >= len(KEYPOINT_NAMES):
                continue
            score = float(confidences[point_index]) if point_index < len(confidences) else 1.0
            if score <= 0.05:
                continue
            x = float(coordinates_xy[0])
            y = float(coordinates_xy[1])
            if x <= 0 and y <= 0:
                continue
            points[KEYPOINT_NAMES[point_index]] = {"x": x, "y": y, "score": score}

    for start_name, end_name in COCO_SKELETON:
        start = points.get(start_name)
        end = points.get(end_name)
        if not start or not end:

```

```

        continue
    skeleton.append(
        {
            "name": f"person_{pose_index}_{start_name}_to_{end_name}",
            "from": f"person_{pose_index}_{start_name}",
            "to": f"person_{pose_index}_{end_name}",
            "x1": start["x"],
            "y1": start["y"],
            "x2": end["x"],
            "y2": end["y"],
            "score": min(start["score"], end["score"]),
        }
    )
return skeleton

```

```

def _annotation_from_result(task_type: str, result_json: dict) -> dict:
    if task_type == "detection":
        return {"shapes": result_json.get("boxes", [])}
    if task_type == "segmentation":
        return {"shapes": result_json.get("segments", [])}
    if task_type == "pose":
        return {"keypoints": result_json.get("keypoints", []), "skeleton": result_json.get("skeleton", [])}
    if task_type == "classification":
        return {"labels": result_json.get("labels", [])}
    return {"description": result_json.get("description", "")}

```

backend/app/api/annotations.py

```

from fastapi import APIRouter, Depends, HTTPException

from app.core.dependencies import get_store
from app.models.schemas import AnnotationsDocument, ReviewRequest, now_utc
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/assets", tags=["annotations"])

@router.get("/{asset_id}/annotations", response_model=AnnotationsDocument)
def get_annotations(asset_id: str, store: PostgresStore = Depends(get_store)) -> AnnotationsDocument:
    annotations = store.get_annotations(asset_id)
    if annotations is None:
        raise HTTPException(status_code=404, detail="annotations not found")
    return annotations

@router.put("/{asset_id}/annotations", response_model=AnnotationsDocument)
def save_annotations(
    asset_id: str,
    annotations: AnnotationsDocument,
    store: PostgresStore = Depends(get_store),
) -> AnnotationsDocument:
    if store.get_asset(asset_id) is None:
        raise HTTPException(status_code=404, detail="asset not found")
    if annotations.asset_id != asset_id:
        raise HTTPException(status_code=400, detail="asset_id mismatch")
    annotations.updated_at = now_utc()
    return store.save_annotations(annotations)

@router.post("/{asset_id}/annotations/review", response_model=AnnotationsDocument)
def review_annotations(
    asset_id: str,
    review: ReviewRequest,
    store: PostgresStore = Depends(get_store),
) -> AnnotationsDocument:
    annotations = store.get_annotations(asset_id)
    if annotations is None:

```

```

        raise HTTPException(status_code=404, detail="annotations not found")
    annotations.review_status = review.status
    for frame in annotations.frames:
        frame.review_status = review.status
        for obj in frame.objects:
            if review.status == "approved" and obj.status == "auto":
                obj.status = "confirmed"
            elif review.status == "rejected":
                obj.status = "rejected"
    annotations.reviewed_at = now_utc() if review.status in {"approved", "rejected"} else None
    annotations.updated_at = now_utc()
    return store.save_annotations(annotations)

```

backend/app/api/assets.py

```

from fastapi import APIRouter, Depends, File, HTTPException, UploadFile
from fastapi.responses import FileResponse

from app.core.dependencies import get_storage_service, get_store
from app.models.schemas import Asset
from app.repositories.postgres_store import PostgresStore
from app.services.storage_service import StorageService

router = APIRouter(prefix="/api/assets", tags=["assets"])

@router.post("/upload", response_model=Asset)
def upload_asset(
    file: UploadFile = File(...),
    storage: StorageService = Depends(get_storage_service),
    store: PostgresStore = Depends(get_store),
) -> Asset:
    asset = storage.save_upload(file)
    return store.create_asset(asset)

@router.get("", response_model=list[Asset])
def list_assets(store: PostgresStore = Depends(get_store)) -> list[Asset]:
    return store.list_assets()

@router.get("/{asset_id}", response_model=Asset)
def get_asset(asset_id: str, store: PostgresStore = Depends(get_store)) -> Asset:
    asset = store.get_asset(asset_id)
    if asset is None:
        raise HTTPException(status_code=404, detail="asset not found")
    return asset

@router.get("/{asset_id}/frames/{frame_index}/image")
def get_frame_image(
    asset_id: str,
    frame_index: int,
    storage: StorageService = Depends(get_storage_service),
    store: PostgresStore = Depends(get_store),
) -> FileResponse:
    if store.get_asset(asset_id) is None:
        raise HTTPException(status_code=404, detail="asset not found")
    frame_path = storage.frame_path(asset_id, frame_index)
    if not frame_path.exists():
        raise HTTPException(status_code=404, detail="frame image not found")
    return FileResponse(frame_path, media_type="image/jpeg")

```

backend/app/api/auth.py

```

from datetime import timedelta

```

```

from fastapi import APIRouter, Depends, HTTPException, Request, Response

from app.core.dependencies import get_store
from app.core.security import hash_password, hash_session_token, new_session_token, verify_password
from app.models.schemas import ApiResponse, AuthLoginRequest, AuthRegisterRequest, AuthUser,
AuthUserRoleUpdateRequest, now_utc
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/auth", tags=["auth"])
AUTH_COOKIE_NAME = "railway_session"
SESSION_MAX_AGE_SECONDS = 7 * 24 * 60 * 60

def resolve_user(request: Request, store: PostgresStore) -> AuthUser | None:
    token = request.cookies.get(AUTH_COOKIE_NAME, "")
    if not token:
        return None
    return store.get_user_by_auth_session(hash_session_token(token))

def require_user(request: Request, store: PostgresStore = Depends(get_store)) -> AuthUser:
    user = getattr(request.state, "current_user", None) or resolve_user(request, store)
    if user is None:
        raise HTTPException(status_code=401, detail="登录状态已失效, 请重新登录")
    return user

def require_reviewer(current_user: AuthUser = Depends(require_user)) -> AuthUser:
    if current_user.role not in {"reviewer", "admin"}:
        raise HTTPException(status_code=403, detail="该操作需要审核员或管理员权限")
    return current_user

def require_admin(current_user: AuthUser = Depends(require_user)) -> AuthUser:
    if current_user.role != "admin":
        raise HTTPException(status_code=403, detail="该操作需要管理员权限")
    return current_user

@router.post("/register", response_model=ApiResponse)
def register(
    request: AuthRegisterRequest,
    response: Response,
    store: PostgresStore = Depends(get_store),
) -> ApiResponse:
    user = store.create_user(request.username, request.display_name, hash_password(request.password))
    if user is None:
        raise HTTPException(status_code=409, detail="该账号已存在")
    _start_session(response, user, store)
    return ApiResponse(data=_user_payload(user))

@router.post("/login", response_model=ApiResponse)
def login(
    request: AuthLoginRequest,
    response: Response,
    store: PostgresStore = Depends(get_store),
) -> ApiResponse:
    credentials = store.get_user_credentials(request.username)
    if credentials is None or not verify_password(request.password, credentials[1]) or not
credentials[0].is_active:
        raise HTTPException(status_code=401, detail="账号或密码错误")
    user = credentials[0]
    _start_session(response, user, store)
    return ApiResponse(data=_user_payload(user))

@router.get("/me", response_model=ApiResponse)
def current_user(request: Request, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    user = resolve_user(request, store)

```

```

    return ApiResponse(data=_user_payload(user) if user else None)

@router.post("/logout", response_model=ApiResponse)
def logout(request: Request, response: Response, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    token = request.cookies.get(AUTH_COOKIE_NAME, "")
    if token:
        store.delete_auth_session(hash_session_token(token))
        response.delete_cookie(AUTH_COOKIE_NAME, path="/", httponly=True, samesite="lax")
    return ApiResponse(data={"loggedOut": True})

@router.get("/users", response_model=ApiResponse)
def list_users(
    store: PostgresStore = Depends(get_store),
    _: AuthUser = Depends(require_admin),
) -> ApiResponse:
    return ApiResponse(data=[_user_payload(user) for user in store.list_users()])

@router.put("/users/{user_id}/role", response_model=ApiResponse)
def update_user_role(
    user_id: str,
    request: AuthUserRoleUpdateRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_admin),
) -> ApiResponse:
    if user_id == current_user.user_id and request.role != "admin":
        raise HTTPException(status_code=400, detail="不能移除自己的管理员权限")
    user = store.update_user_role(user_id, request.role)
    if user is None:
        raise HTTPException(status_code=404, detail="用户不存在")
    return ApiResponse(data=_user_payload(user))

def _start_session(response: Response, user: AuthUser, store: PostgresStore) -> None:
    token = new_session_token()
    store.create_auth_session(
        hash_session_token(token),
        user.user_id,
        now_utc() + timedelta(seconds=SESSION_MAX_AGE_SECONDS),
    )
    response.set_cookie(
        AUTH_COOKIE_NAME,
        token,
        max_age=SESSION_MAX_AGE_SECONDS,
        httponly=True,
        samesite="lax",
        secure=False,
        path="/",
    )

def _user_payload(user: AuthUser) -> dict:
    return {
        "userId": user.user_id,
        "username": user.username,
        "displayName": user.display_name,
        "role": user.role,
    }

```

backend/app/api/export.py

```

from typing import Literal

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import JSONResponse, PlainTextResponse

from app.core.dependencies import get_export_service

```

```

from app.services.export_service import ExportService

router = APIRouter(prefix="/api/assets", tags=["export"])

@router.get("/{asset_id}/export")
def export_annotations(
    asset_id: str,
    format: Literal["json", "coco", "yolo"] = "json",
    export_service: ExportService = Depends(get_export_service),
):
    annotations = export_service.get_annotations(asset_id)
    if annotations is None:
        raise HTTPException(status_code=404, detail="annotations not found")
    if format == "json":
        return JSONResponse(annotations.model_dump(mode="json"))
    if format == "coco":
        return JSONResponse(export_service.to_coco(annotations))
    return PlainTextResponse(export_service.to_yolo_text(annotations), media_type="text/plain")

```

backend/app/api/image_tasks.py

```

from fastapi import APIRouter, Depends, HTTPException, Query

from app.api.auth import require_reviewer, require_user
from app.core.dependencies import get_store
from app.models.schemas import (
    ApiResponse,
    AuthUser,
    ImageTaskAnnotationUpdateRequest,
    ImageTaskCreateRequest,
    ImageTaskResult,
    ImageTaskResultSaveRequest,
    ImageTaskReviewRequest,
    ImageTaskResultVersion,
    ImageTaskType,
)
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/image-tasks", tags=["image-workspace"])

@router.post("/create", response_model=ApiResponse)
def create_image_task(
    request: ImageTaskCreateRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_edit_permission(current_user)
    image = store.get_accessible_image_asset(request.image_id, current_user.user_id)
    if image is None:
        raise HTTPException(status_code=404, detail="当前图片不存在")
    task = store.create_or_get_image_task(
        request.image_id,
        request.task_type,
        request.session_id or "default",
        current_user.user_id,
    )
    return ApiResponse(data=_task_payload(task))

@router.post("/result/save", response_model=ApiResponse)
def save_image_task_result(
    request: ImageTaskResultSaveRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:

```

```

    _require_edit_permission(current_user)
    if store.get_accessible_image_asset(request.image_id, current_user.user_id) is None:
        raise HTTPException(status_code=404, detail="当前图片不存在")
    _ensure_result_editable(store, request.task_id)
    result = ImageTaskResult(
        task_id=request.task_id,
        image_id=request.image_id,
        task_type=request.task_type,
        result_image_url=request.result_image_url,
        result_image_path=request.result_image_path,
        result_json=request.result_json,
        annotation_json=request.annotation_json,
        description_text=request.description_text,
        source=request.source,
        model_id=request.model_id,
    )
    saved, version = store.save_image_task_result(result)
    return ApiResponse(data={"resultId": saved.result_id, "versionId": version.version_id, "versionNo":
version.version_no})

```

```

@router.get("/result/versions", response_model=ApiResponse)
def list_image_task_result_versions(
    image_id: str = Query(alias="imageId"),
    task_type: ImageTaskType = Query(alias="taskType"),
    session_id: str = Query(default="default", alias="sessionId"),
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_image_access(store, image_id, current_user)
    versions = store.list_image_task_result_versions(image_id, task_type, session_id or "default")
    return ApiResponse(data=[_version_payload(version) for version in versions])

```

```

@router.get("/result/version/{version_id}", response_model=ApiResponse)
def get_image_task_result_version(
    version_id: str,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    version = store.get_image_task_result_version(version_id)
    if version is None:
        raise HTTPException(status_code=404, detail="历史版本不存在")
    _require_image_access(store, version.image_id, current_user)
    return ApiResponse(data=_version_payload(version))

```

```

@router.post("/result/version/{version_id}/restore", response_model=ApiResponse)
def restore_image_task_result_version(
    version_id: str,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_edit_permission(current_user)
    existing = store.get_image_task_result_version(version_id)
    if existing is None:
        raise HTTPException(status_code=404, detail="历史版本不存在")
    _require_image_access(store, existing.image_id, current_user)
    _ensure_result_editable(store, existing.task_id)
    restored = store.restore_image_task_result_version(version_id)
    if restored is None:
        raise HTTPException(status_code=404, detail="历史版本不存在")
    result, version = restored
    return ApiResponse(data={*_result_payload(result), "versionId": version.version_id, "versionNo":
version.version_no})

```

```

@router.get("/result", response_model=ApiResponse)
def get_image_task_result(
    image_id: str = Query(alias="imageId"),
    task_type: ImageTaskType = Query(alias="taskType"),

```

```

    session_id: str = Query(default="default", alias="sessionId"),
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_image_access(store, image_id, current_user)
    result = store.get_image_task_result(image_id, task_type, session_id or "default")
    if result is None:
        return ApiResponse(message="no result", data=None)
    return ApiResponse(data=_result_payload(result))

@router.put("/annotation", response_model=ApiResponse)
def update_image_task_annotation(
    request: ImageTaskAnnotationUpdateRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    _require_edit_permission(current_user)
    _require_image_access(store, request.image_id, current_user)
    _ensure_result_editable(store, request.task_id)
    result = store.update_image_task_annotation(
        request.task_id,
        request.image_id,
        request.task_type,
        request.annotation_json,
    )
    if result is None:
        raise HTTPException(status_code=404, detail="当前任务结果不存在")
    return ApiResponse(data=_result_payload(result))

@router.post("/result/{task_id}/submit", response_model=ApiResponse)
def submit_image_task_result(
    task_id: str,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    result = store.submit_image_task_result(task_id, current_user.user_id)
    if result is None:
        raise HTTPException(status_code=409, detail="任务不存在、不属于当前用户或当前状态不能提交")
    return ApiResponse(data=_result_payload(result))

@router.post("/result/{task_id}/review", response_model=ApiResponse)
def review_image_task_result(
    task_id: str,
    request: ImageTaskReviewRequest,
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_reviewer),
) -> ApiResponse:
    result = store.review_image_task_result(task_id, request.status, request.comment, current_user.user_id)
    if result is None:
        raise HTTPException(status_code=409, detail="只有其他用户提交的待审核任务可以执行审核")
    return ApiResponse(data=_result_payload(result))

def _require_image_access(store: PostgresStore, image_id: str, current_user: AuthUser) -> None:
    image = (
        store.get_image_asset(image_id)
        if current_user.role in {"reviewer", "admin"}
        else store.get_accessible_image_asset(image_id, current_user.user_id)
    )
    if image is None:
        raise HTTPException(status_code=404, detail="当前图片不存在或无权访问")

def _require_edit_permission(current_user: AuthUser) -> None:
    if current_user.role == "reviewer":
        raise HTTPException(status_code=403, detail="审核员只能查看和审核任务，不能修改标注")

```

```

def _ensure_result_editable(store: PostgresStore, task_id: str) -> None:
    result = store.get_image_task_result_by_task_id(task_id)
    if result is not None and result.review_status == "approved":
        raise HTTPException(status_code=409, detail="审核通过的任务已经锁定, 不能继续修改")

def _task_payload(task) -> dict:
    return {
        "taskId": task.task_id,
        "imageId": task.image_id,
        "taskType": task.task_type,
        "status": task.status,
        "createdAt": task.created_at.isoformat(),
        "updatedAt": task.updated_at.isoformat(),
    }

def _result_payload(result: ImageTaskResult) -> dict:
    return {
        "resultId": result.result_id,
        "taskId": result.task_id,
        "imageId": result.image_id,
        "taskType": result.task_type,
        "status": result.status,
        "resultImageUrl": result.result_image_url,
        "resultImagePath": result.result_image_path,
        "resultJson": result.result_json,
        "annotationJson": result.annotation_json,
        "descriptionText": result.description_text,
        "source": result.source,
        "modelId": result.model_id,
        "latestVersionId": result.latest_version_id,
        "latestVersionNo": result.latest_version_no,
        "reviewStatus": result.review_status,
        "submittedBy": result.submitted_by,
        "submittedAt": result.submitted_at.isoformat() if result.submitted_at else None,
        "reviewedBy": result.reviewed_by,
        "reviewedAt": result.reviewed_at.isoformat() if result.reviewed_at else None,
        "reviewComment": result.review_comment,
        "createdAt": result.created_at.isoformat(),
        "updatedAt": result.updated_at.isoformat(),
    }

def _version_payload(version: ImageTaskResultVersion) -> dict:
    return {
        "versionId": version.version_id,
        "resultId": version.result_id,
        "taskId": version.task_id,
        "imageId": version.image_id,
        "taskType": version.task_type,
        "versionNo": version.version_no,
        "source": version.source,
        "modelId": version.model_id,
        "resultImageUrl": version.result_image_url,
        "resultImagePath": version.result_image_path,
        "resultJson": version.result_json,
        "annotationJson": version.annotation_json,
        "descriptionText": version.description_text,
        "createdAt": version.created_at.isoformat(),
    }

```

backend/app/api/images.py

```

import shutil
from pathlib import Path
from uuid import uuid4

from fastapi import APIRouter, Depends, File, Form, HTTPException, Query, UploadFile

```

```

from PIL import Image

from app.core.config import Settings, get_settings
from app.core.dependencies import get_store
from app.api.auth import require_user
from app.models.schemas import ApiResponse, AuthUser, ImageAsset, ImageHistoryResponse, ImageTaskType, now_utc
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/images", tags=["image-workspace"])

SUPPORTED_IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp", ".bmp"}
MAX_IMAGE_BYTES = 20 * 1024 * 1024

@router.post("/upload", response_model=ApiResponse)
def upload_image(
    file: UploadFile = File(...),
    task_type: ImageTaskType = Form(..., alias="taskType"),
    session_id: str = Form(default="default", alias="sessionId"),
    settings: Settings = Depends(get_settings),
    store: PostgresStore = Depends(get_store),
    current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    del task_type
    original_name = Path(file.filename or "upload.png").name
    extension = Path(original_name).suffix.lower()
    if extension not in SUPPORTED_IMAGE_EXTENSIONS:
        raise HTTPException(status_code=400, detail="图片格式不支持，请上传 jpg、jpeg、png、webp 或 bmp")

    current = now_utc()
    target_dir = settings.upload_dir / "images" / "original" / f"{current.year:04d}" / f"{current.month:02d}"
    target_dir.mkdir(parents=True, exist_ok=True)
    file_name = f"{uuid4().hex}{extension}"
    target_path = target_dir / file_name

    with target_path.open("wb") as handle:
        shutil.copyfileobj(file.file, handle)

    file_size = target_path.stat().st_size
    if file_size > MAX_IMAGE_BYTES:
        target_path.unlink(missing_ok=True)
        raise HTTPException(status_code=400, detail="图片大小超过 20MB 限制")

    try:
        with Image.open(target_path) as image:
            width, height = image.size
            image.verify()
    except Exception as exc:
        target_path.unlink(missing_ok=True)
        raise HTTPException(status_code=400, detail="图片文件无法解析") from exc

    relative_url = "/" + target_path.relative_to(settings.data_dir).as_posix()
    image_asset = ImageAsset(
        original_name=original_name,
        file_name=file_name,
        file_url=relative_url,
        file_path=str(target_path),
        mime_type=file.content_type or "application/octet-stream",
        file_size=file_size,
        width=width,
        height=height,
        user_id=current_user.user_id,
        session_id=session_id or "default",
    )
    store.create_image_asset(image_asset)
    return ApiResponse(data=_image_payload(image_asset))

@router.get("/list", response_model=ApiResponse)
def list_images(

```

```

task_type: ImageTaskType | None = Query(default=None, alias="taskType"),
page: int = Query(default=1, ge=1),
page_size: int = Query(default=50, ge=1, le=200, alias="pageSize"),
session_id: str | None = Query(default=None, alias="sessionId"),
store: PostgresStore = Depends(get_store),
current_user: AuthUser = Depends(require_user),
) -> ApiResponse:
    records, total = store.list_image_assets(
        task_type=task_type,
        session_id=session_id,
        user_id=None if current_user.role in {"reviewer", "admin"} else current_user.user_id,
        page=page,
        page_size=page_size,
    )
    data = ImageHistoryResponse(records=records, total=total)
    return ApiResponse(data={"records": [_image_payload(record) for record in data.records], "total": data.total})

def _image_payload(image: ImageAsset) -> dict:
    return {
        "imageId": image.image_id,
        "imageUrl": image.file_url,
        "originalName": image.original_name,
        "fileName": image.file_name,
        "fileUrl": image.file_url,
        "filePath": image.file_path,
        "mimeType": image.mime_type,
        "fileSize": image.file_size,
        "width": image.width,
        "height": image.height,
        "sessionId": image.session_id,
        "createdAt": image.created_at.isoformat(),
        "updatedAt": image.updated_at.isoformat(),
        "hasCurrentTaskResult": image.has_current_task_result,
        "taskStatus": image.task_status,
    }

```

backend/app/api/labels.py

```

from fastapi import APIRouter, Depends, HTTPException

from app.api.auth import require_admin
from app.core.dependencies import get_store
from app.models.schemas import ApiResponse, AuthUser, LabelConfigCreateRequest, LabelConfigUpdateRequest
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/labels", tags=["labels"])

@router.get("")
def list_labels(store: PostgresStore = Depends(get_store)) -> ApiResponse:
    labels = store.list_label_configs()
    return ApiResponse(data=[label.model_dump(mode="json", by_alias=True) for label in labels])

@router.post("")
def create_label(request: LabelConfigCreateRequest, store: PostgresStore = Depends(get_store), _: AuthUser = Depends(require_admin)) -> ApiResponse:
    try:
        label = store.create_label_config(
            english_name=request.english_name,
            chinese_name=request.chinese_name,
            description=request.description,
            copy_from_label_id=request.copy_from_label_id,
        )
    except Exception as exc:
        raise HTTPException(status_code=409, detail="标签数据不合法") from exc
    return ApiResponse(data=label.model_dump(mode="json", by_alias=True))

```

```

@router.post("/{label_id}/copy")
def copy_label(label_id: int, store: PostgresStore = Depends(get_store), _: AuthUser = Depends(require_admin)) ->
ApiResponse:
    label = store.copy_label_config(label_id)
    if label is None:
        raise HTTPException(status_code=404, detail="标签不存在")
    return ApiResponse(data=label.model_dump(mode="json", by_alias=True))

@router.put("/{label_id}")
def update_label(
    label_id: int,
    request: LabelConfigUpdateRequest,
    store: PostgresStore = Depends(get_store),
    _: AuthUser = Depends(require_admin),
) -> ApiResponse:
    try:
        label = store.update_label_config(
            label_id=label_id,
            english_name=request.english_name,
            chinese_name=request.chinese_name,
            description=request.description,
        )
    except Exception as exc:
        raise HTTPException(status_code=409, detail="标签数据不合法") from exc
    if label is None:
        raise HTTPException(status_code=404, detail="标签不存在")
    return ApiResponse(data=label.model_dump(mode="json", by_alias=True))

@router.delete("/{label_id}")
def delete_label(label_id: int, store: PostgresStore = Depends(get_store), _: AuthUser = Depends(require_admin)) ->
ApiResponse:
    deleted = store.delete_label_config(label_id)
    if not deleted:
        raise HTTPException(status_code=404, detail="标签不存在")
    return ApiResponse(data={"deleted": True})

```

backend/app/api/tasks.py

```

from fastapi import APIRouter, Depends, HTTPException

from app.core.config import Settings, get_settings
from app.core.dependencies import get_store, get_task_service
from app.models.schemas import DetectTask, DetectionRequest
from app.repositories.postgres_store import PostgresStore
from app.services.task_service import TaskService

router = APIRouter(prefix="/api/tasks", tags=["tasks"])

@router.post("/detect", response_model=DetectTask, status_code=202)
def create_detection_task(
    request: DetectionRequest,
    task_service: TaskService = Depends(get_task_service),
    settings: Settings = Depends(get_settings),
    store: PostgresStore = Depends(get_store),
) -> DetectTask:
    if store.get_asset(request.asset_id) is None:
        raise HTTPException(status_code=404, detail="asset not found")
    task = task_service.create_detection_task(request)
    if settings.run_tasks_inline:
        task_service.run_detection(task.id)
    return store.get_task(task.id) or task
    return task

```

```

@router.get("", response_model=list[DetectTask])
def list_tasks(store: PostgresStore = Depends(get_store)) -> list[DetectTask]:
    return store.list_tasks()

@router.get("/{task_id}", response_model=DetectTask)
def get_task(task_id: str, store: PostgresStore = Depends(get_store)) -> DetectTask:
    task = store.get_task(task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="task not found")
    return task

```

backend/app/api/video_captions.py

```

from pathlib import Path

from fastapi import APIRouter, Depends, HTTPException, Query
from fastapi.responses import FileResponse

from app.core.dependencies import get_store
from app.models.schemas import ApiResponse, VideoCaptionBatch, VideoCaptionFrame, VideoCaptionVideo
from app.repositories.postgres_store import PostgresStore

router = APIRouter(prefix="/api/video-captions", tags=["video-captions"])

@router.get("/batches", response_model=ApiResponse)
def list_batches(store: PostgresStore = Depends(get_store)) -> ApiResponse:
    return ApiResponse(data=[_batch_payload(batch) for batch in store.list_video_caption_batches()])

@router.get("/batches/{batch_id}", response_model=ApiResponse)
def get_batch(batch_id: str, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    batch = store.get_video_caption_batch(batch_id)
    if batch is None:
        raise HTTPException(status_code=404, detail="video caption batch not found")
    return ApiResponse(data=_batch_payload(batch))

@router.get("/batches/{batch_id}/videos", response_model=ApiResponse)
def list_videos(batch_id: str, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    if store.get_video_caption_batch(batch_id) is None:
        raise HTTPException(status_code=404, detail="video caption batch not found")
    return ApiResponse(data=[_video_payload(video) for video in store.list_video_caption_videos(batch_id)])

@router.get("/videos/{video_id}/frames", response_model=ApiResponse)
def list_frames(
    video_id: str,
    page: int = Query(default=1, ge=1),
    page_size: int = Query(default=100, ge=1, le=500, alias="pageSize"),
    query: str = Query(default=""),
    store: PostgresStore = Depends(get_store),
) -> ApiResponse:
    if store.get_video_caption_video(video_id) is None:
        raise HTTPException(status_code=404, detail="video caption video not found")
    frames, total = store.list_video_caption_frames(video_id, page=page, page_size=page_size, query=query.strip())
    return ApiResponse(data={"records": [_frame_payload(frame) for frame in frames], "total": total})

@router.get("/frames/{frame_id}", response_model=ApiResponse)
def get_frame(frame_id: str, store: PostgresStore = Depends(get_store)) -> ApiResponse:
    frame = store.get_video_caption_frame(frame_id)
    if frame is None:
        raise HTTPException(status_code=404, detail="video caption frame not found")
    return ApiResponse(data=_frame_payload(frame))

```

```

@router.get("/frames/{frame_id}/image")
def get_frame_image(frame_id: str, store: PostgresStore = Depends(get_store)) -> FileResponse:
    frame = store.get_video_caption_frame(frame_id)
    if frame is None:
        raise HTTPException(status_code=404, detail="video caption frame not found")
    image_path = Path(frame.image_path)
    if not image_path.exists() or not image_path.is_file():
        raise HTTPException(status_code=404, detail="video caption frame image not found")
    return FileResponse(image_path, media_type="image/jpeg")

def _batch_payload(batch: VideoCaptionBatch) -> dict:
    return {
        "batchId": batch.batch_id,
        "name": batch.name,
        "sourceDir": batch.source_dir,
        "outputDir": batch.output_dir,
        "modelId": batch.model_id,
        "prompt": batch.prompt,
        "frameIntervalSeconds": batch.frame_interval_seconds,
        "status": batch.status,
        "totalVideos": batch.total_videos,
        "totalFrames": batch.total_frames,
        "metadata": batch.metadata,
        "createdAt": batch.created_at.isoformat(),
        "updatedAt": batch.updated_at.isoformat(),
    }

def _video_payload(video: VideoCaptionVideo) -> dict:
    return {
        "videoId": video.video_id,
        "batchId": video.batch_id,
        "filename": video.filename,
        "displayName": video.display_name or video.filename,
        "keywords": video.keywords,
        "sourcePath": video.source_path,
        "frameDir": video.frame_dir,
        "fps": video.fps,
        "width": video.width,
        "height": video.height,
        "frameCount": video.frame_count,
        "durationMs": video.duration_ms,
        "status": video.status,
        "error": video.error,
        "createdAt": video.created_at.isoformat(),
        "updatedAt": video.updated_at.isoformat(),
    }

def _frame_payload(frame: VideoCaptionFrame) -> dict:
    return {
        "frameId": frame.frame_id,
        "batchId": frame.batch_id,
        "videoId": frame.video_id,
        "frameIndex": frame.frame_index,
        "timestampMs": frame.timestamp_ms,
        "imagePath": frame.image_path,
        "imageUrl": frame.image_url or f"/api/video-captions/frames/{frame.frame_id}/image",
        "width": frame.width,
        "height": frame.height,
        "descriptionText": frame.description_text,
        "modelId": frame.model_id,
        "status": frame.status,
        "error": frame.error,
        "resultJson": frame.result_json,
        "createdAt": frame.created_at.isoformat(),
        "updatedAt": frame.updated_at.isoformat(),
    }

```

backend/app/api/vision.py

```
from typing import Any

from fastapi import APIRouter, Depends, File, HTTPException, UploadFile

from app.core.dependencies import get_vision_service
from app.services.vision_service import VisionFeature, VisionService

router = APIRouter(prefix="/api/vision", tags=["vision"])

@router.post("/analyze")
def analyze_image(
    feature: VisionFeature,
    model_name: str | None = None,
    file: UploadFile = File(...),
    service: VisionService = Depends(get_vision_service),
) -> dict[str, Any]:
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="only image uploads supported")
    try:
        return service.analyze_upload(file, feature, model_name)
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc
```

backend/app/core/__init__.py

```
"""Core backend settings and dependencies."""
```

backend/app/core/config.py

```
from functools import lru_cache
from pathlib import Path

from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "Railway Multi-Target Recognition API"
    data_dir: Path = Path("./data")
    database_url: str = "postgresql+psycopg://deipss:CHANGE_ME@***"
    inference_backend: str = Field(default="mock", pattern="^(mock|ultralitics|tensorrt)$")
    model_path: str = ""
    deepseek_vl2_model_path: str = "./models/deepseek-vl2-tiny"
    deepseek_vl2_prompt: str = "请用中文简要描述这张图片的主要内容，并列出可见目标。"
    device: str = "0"
    run_tasks_inline: bool = False
    task_worker_enabled: bool = True
    task_poll_interval_seconds: float = Field(default=1.0, gt=0)
    cors_origins: str = ""

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    @property
    def upload_dir(self) -> Path:
        return self.data_dir / "uploads"

    @property
    def allowed_origins(self) -> list[str]:
        return [origin.strip() for origin in self.cors_origins.split(",") if origin.strip()]

    @lru_cache
    def get_settings() -> Settings:
        settings = Settings()
```

```
settings.upload_dir.mkdir(parents=True, exist_ok=True)
settings.data_dir.mkdir(parents=True, exist_ok=True)
return settings
```

backend/app/core/dependencies.py

```
from functools import lru_cache

from app.core.config import get_settings
from app.repositories.postgres_store import PostgresStore
from app.services.export_service import ExportService
from app.services.inference_service import InferenceService
from app.services.storage_service import StorageService
from app.services.task_service import TaskService
from app.services.vision_service import VisionService

@lru_cache
def get_store() -> PostgresStore:
    return PostgresStore(get_settings().database_url)

def get_storage_service() -> StorageService:
    settings = get_settings()
    return StorageService(settings.upload_dir)

def get_inference_service() -> InferenceService:
    return InferenceService(get_settings(), get_storage_service())

def get_task_service() -> TaskService:
    return TaskService(get_store(), get_inference_service())

def get_export_service() -> ExportService:
    return ExportService(get_store())

@lru_cache
def get_vision_service() -> VisionService:
    return VisionService(get_settings())
```

backend/app/core/security.py

```
import hashlib
import hmac
import secrets

def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.scrypt(password.encode("utf-8"), salt=salt, n=2**14, r=8, p=1, dklen=32)
    return f"scrypt$16384$8$1${salt.hex()}${digest.hex()}"

def verify_password(password: str, encoded: str) -> bool:
    try:
        algorithm, n, r, p, salt_hex, digest_hex = encoded.split("$", 5)
        if algorithm != "scrypt":
            return False
        actual = hashlib.scrypt(
            password.encode("utf-8"),
            salt=bytes.fromhex(salt_hex),
            n=int(n),
            r=int(r),
            p=int(p),
            dklen=len(bytes.fromhex(digest_hex)),
        )
        return hmac.compare_digest(actual, digest_hex)
```

```

    )
    return hmac.compare_digest(actual, bytes.fromhex(digest_hex))
except (TypeError, ValueError):
    return False

```

```

def new_session_token() -> str:
    return secrets.token_urlsafe(48)

```

```

def hash_session_token(token: str) -> str:
    return hashlib.sha256(token.encode("utf-8")).hexdigest()

```

backend/app/main.py

```

from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from fastapi.staticfiles import StaticFiles

from app.api import ai, annotations, assets, auth, export, image_tasks, images, labels, tasks, video_captions, vision
from app.core.config import get_settings
from app.core.dependencies import get_store, get_task_service
from app.services.task_worker import DatabaseTaskWorker

def create_app() -> FastAPI:
    settings = get_settings()

    @asynccontextmanager
    async def lifespan(_: FastAPI):
        worker = None
        if settings.task_worker_enabled and not settings.run_tasks_inline:
            worker = DatabaseTaskWorker(
                get_store(),
                get_task_service(),
                settings.task_poll_interval_seconds,
            )
            worker.start()
        try:
            yield
        finally:
            if worker is not None:
                worker.stop()
            get_store().close()

    app = FastAPI(title=settings.app_name, lifespan=lifespan)
    app.include_router(auth.router)

    @app.middleware("http")
    async def require_authenticated_session(request: Request, call_next):
        protected_path = request.url.path.startswith("/api/") or request.url.path.startswith("/uploads/")
        public_auth_path = request.url.path.startswith("/api/auth/")
        if protected_path and not public_auth_path and request.method != "OPTIONS":
            user = auth.resolve_user(request, get_store())
            if user is None:
                return JSONResponse(status_code=401, content={"detail": "登录状态已失效, 请重新登录"})
            request.state.current_user = user
        return await call_next(request)

    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.allowed_origins,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

```

```

)

app.include_router/assets.router)
app.include_router(tasks.router)
app.include_router(annotations.router)
app.include_router(export.router)
app.include_router(vision.router)
app.include_router(images.router)
app.include_router(image_tasks.router)
app.include_router(ai.router)
app.include_router(labels.router)
app.include_router(video_captions.router)
app.mount("/uploads", StaticFiles(directory=settings.upload_dir), name="uploads")

@app.get("/health")
def health() -> dict[str, str]:
    database_status = "ok" if get_store().healthcheck() else "unavailable"
    return {
        "status": "ok",
        "database": database_status,
        "inference_backend": settings.inference_backend,
    }

return app

app = create_app()

```

backend/app/models/__init__.py

```

"""Pydantic schemas and persistence models."""

```

backend/app/models/schemas.py

```

from datetime import datetime, timezone
from typing import Any, Literal
from uuid import uuid4

from pydantic import BaseModel, ConfigDict, Field

AssetType = Literal["image", "video", "unknown"]
TaskStatus = Literal["queued", "running", "completed", "failed"]
ObjectStatus = Literal["auto", "confirmed", "edited", "rejected"]
ObjectSource = Literal["auto", "manual"]
ReviewStatus = Literal["pending_review", "in_review", "approved", "rejected"]
ImageTaskType = Literal["detection", "segmentation", "pose", "classification", "caption"]
ImageTaskStatus = Literal["idle", "pending", "processing", "success", "failed"]
ResultSource = Literal["ai", "manual", "edited"]
ImageReviewStatus = Literal["draft", "pending_review", "approved", "rejected"]
VideoCaptionStatus = Literal["pending", "processing", "success", "failed"]

def new_id(prefix: str) -> str:
    return f"{prefix}_{uuid4().hex[:12]}"

def now_utc() -> datetime:
    return datetime.now(timezone.utc)

class BBox(BaseModel):
    x: float = Field(ge=0)
    y: float = Field(ge=0)
    width: float = Field(gt=0)
    height: float = Field(gt=0)

```

```

class DetectionObject(BaseModel):
    id: str = Field(default_factory=lambda: new_id("obj"))
    label: str
    confidence: float = Field(ge=0, le=1)
    bbox: BBox
    track_id: int | None = None
    source: ObjectSource = "auto"
    status: ObjectStatus = "auto"

class AnnotationFrame(BaseModel):
    frame_index: int = Field(ge=0)
    timestamp_ms: int = Field(ge=0)
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    image_url: str | None = None
    review_status: ReviewStatus = "pending_review"
    objects: list[DetectionObject] = Field(default_factory=list)

class AnnotationsDocument(BaseModel):
    asset_id: str
    type: AssetType
    model: str
    review_status: ReviewStatus = "pending_review"
    frames: list[AnnotationFrame] = Field(default_factory=list)
    updated_at: datetime = Field(default_factory=now_utc)
    reviewed_at: datetime | None = None

class Asset(BaseModel):
    id: str = Field(default_factory=lambda: new_id("asset"))
    filename: str
    content_type: str
    type: AssetType = "unknown"
    path: str
    size_bytes: int = Field(ge=0)
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    frame_count: int | None = Field(default=None, ge=1)
    duration_ms: int | None = Field(default=None, ge=0)
    fps: float | None = Field(default=None, gt=0)
    created_at: datetime = Field(default_factory=now_utc)

class DetectionRequest(BaseModel):
    asset_id: str
    confidence: float = Field(default=0.25, ge=0, le=1)
    iou: float = Field(default=0.7, ge=0, le=1)
    frame_stride: int = Field(default=1, ge=1)
    max_frames: int | None = Field(default=None, ge=1)
    model_name: str | None = None

class ReviewRequest(BaseModel):
    status: ReviewStatus

class DetectTask(BaseModel):
    id: str = Field(default_factory=lambda: new_id("task"))
    asset_id: str
    status: TaskStatus = "queued"
    progress: float = Field(default=0, ge=0, le=1)
    model_name: str = "mock"
    params: DetectionRequest
    error: str | None = None
    created_at: datetime = Field(default_factory=now_utc)
    started_at: datetime | None = None
    finished_at: datetime | None = None

```

```
class ImageAsset(BaseModel):
    image_id: str = Field(default_factory=lambda: new_id("img"))
    original_name: str
    file_name: str
    file_url: str
    file_path: str
    mime_type: str | None = None
    file_size: int = Field(ge=0)
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    user_id: str | None = None
    session_id: str = "default"
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)
    has_current_task_result: bool = False
    task_status: ImageTaskStatus = "idle"
```

```
class ImageHistoryResponse(BaseModel):
    records: List[ImageAsset]
    total: int
```

```
class ImageTask(BaseModel):
    task_id: str = Field(default_factory=lambda: new_id("itask"))
    image_id: str
    task_type: ImageTaskType
    status: ImageTaskStatus = "pending"
    user_id: str | None = None
    session_id: str = "default"
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)
```

```
class ImageTaskCreateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    image_id: str = Field(alias="imageId")
    task_type: ImageTaskType = Field(alias="taskType")
    session_id: str | None = Field(default=None, alias="sessionId")
    user_id: str | None = Field(default=None, alias="userId")
```

```
class ImageTaskResult(BaseModel):
    result_id: str = Field(default_factory=lambda: new_id("ires"))
    task_id: str
    image_id: str
    task_type: ImageTaskType
    status: ImageTaskStatus = "success"
    result_image_url: str = ""
    result_image_path: str = ""
    result_json: dict[str, Any] | None = None
    annotation_json: dict[str, Any] | None = None
    description_text: str = ""
    source: ResultSource = "edited"
    model_id: str = ""
    latest_version_id: str | None = None
    latest_version_no: int | None = None
    review_status: ImageReviewStatus = "draft"
    submitted_by: str | None = None
    submitted_at: datetime | None = None
    reviewed_by: str | None = None
    reviewed_at: datetime | None = None
    review_comment: str = ""
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)
```

```
class ImageTaskResultSaveRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)
```

```

task_id: str = Field(alias="taskId")
image_id: str = Field(alias="imageId")
task_type: ImageTaskType = Field(alias="taskType")
result_image_url: str = Field(default="", alias="resultImageUrl")
result_image_path: str = Field(default="", alias="resultImagePath")
result_json: dict[str, Any] | None = Field(default=None, alias="resultJson")
annotation_json: dict[str, Any] | None = Field(default=None, alias="annotationJson")
description_text: str = Field(default="", alias="descriptionText")
source: ResultSource = "edited"
model_id: str = Field(default="", alias="modelId")

```

```

class ImageTaskResultVersion(BaseModel):
    version_id: str = Field(default_factory=lambda: new_id("iver"))
    result_id: str
    task_id: str
    image_id: str
    task_type: ImageTaskType
    version_no: int = Field(ge=1)
    source: ResultSource = "edited"
    model_id: str = ""
    result_image_url: str = ""
    result_image_path: str = ""
    result_json: dict[str, Any] | None = None
    annotation_json: dict[str, Any] | None = None
    description_text: str = ""
    created_at: datetime = Field(default_factory=now_utc)

```

```

class ImageTaskAnnotationUpdateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    task_id: str = Field(alias="taskId")
    image_id: str = Field(alias="imageId")
    task_type: ImageTaskType = Field(alias="taskType")
    annotation_json: dict[str, Any] = Field(alias="annotationJson")

```

```

class ImageTaskReviewRequest(BaseModel):
    status: Literal["approved", "rejected"]
    comment: str = Field(default="", max_length=1000)

```

```

class AuthUser(BaseModel):
    user_id: str = Field(default_factory=lambda: new_id("user"), alias="userId")
    username: str
    display_name: str = Field(alias="displayName")
    role: str = "student"
    is_active: bool = Field(default=True, alias="isActive")
    created_at: datetime = Field(default_factory=now_utc, alias="createdAt")
    updated_at: datetime = Field(default_factory=now_utc, alias="updatedAt")

    model_config = ConfigDict(populate_by_name=True)

```

```

class AuthRegisterRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    username: str = Field(min_length=3, max_length=50, pattern=r"^[A-Za-z0-9_.-]+$")
    display_name: str = Field(min_length=1, max_length=80, alias="displayName")
    password: str = Field(min_length=6, max_length=128)

```

```

class AuthLoginRequest(BaseModel):
    username: str = Field(min_length=1, max_length=50)
    password: str = Field(min_length=1, max_length=128)

```

```

class AuthUserRoleUpdateRequest(BaseModel):
    role: Literal["student", "reviewer", "admin"]

```

```

class AiInferRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    image_id: str = Field(alias="imageId")
    task_type: ImageTaskType = Field(alias="taskType")
    model_name: str | None = Field(default=None, alias="modelName")
    session_id: str | None = Field(default=None, alias="sessionId")

class LabelConfig(BaseModel):
    label_id: int = Field(ge=0, alias="labelId")
    english_name: str = Field(min_length=1, max_length=120, alias="englishName")
    chinese_name: str = Field(min_length=1, max_length=120, alias="chineseName")
    description: str = ""
    created_at: datetime = Field(default_factory=now_utc, alias="createdAt")
    updated_at: datetime = Field(default_factory=now_utc, alias="updatedAt")

    model_config = ConfigDict(populate_by_name=True)

class LabelConfigCreateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    english_name: str = Field(min_length=1, max_length=120, alias="englishName")
    chinese_name: str = Field(min_length=1, max_length=120, alias="chineseName")
    description: str = ""
    copy_from_label_id: int | None = Field(default=None, ge=0, alias="copyFromLabelId")

class LabelConfigUpdateRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    english_name: str = Field(min_length=1, max_length=120, alias="englishName")
    chinese_name: str = Field(min_length=1, max_length=120, alias="chineseName")
    description: str = ""

class ApiResponse(BaseModel):
    code: int = 200
    message: str = "success"
    data: Any = None

class ErrorResponse(BaseModel):
    detail: str

class VideoCaptionBatch(BaseModel):
    batch_id: str = Field(default_factory=lambda: new_id("vcbatch"))
    name: str = Field(min_length=1, max_length=160)
    source_dir: str = ""
    output_dir: str = ""
    model_id: str = "deepseek-ai/deepseek-vl2-tiny"
    prompt: str = ""
    frame_interval_seconds: float = Field(default=1.0, gt=0)
    status: VideoCaptionStatus = "pending"
    total_videos: int = Field(default=0, ge=0)
    total_frames: int = Field(default=0, ge=0)
    metadata: dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)

class VideoCaptionVideo(BaseModel):
    video_id: str = Field(default_factory=lambda: new_id("vcvideo"))
    batch_id: str
    filename: str
    display_name: str = ""
    keywords: list[str] = Field(default_factory=list)
    source_path: str

```

```

frame_dir: str = ""
fps: float | None = Field(default=None, gt=0)
width: int | None = Field(default=None, ge=1)
height: int | None = Field(default=None, ge=1)
frame_count: int | None = Field(default=None, ge=1)
duration_ms: int | None = Field(default=None, ge=0)
status: VideoCaptionStatus = "pending"
error: str = ""
created_at: datetime = Field(default_factory=now_utc)
updated_at: datetime = Field(default_factory=now_utc)

```

```

class VideoCaptionFrame(BaseModel):
    frame_id: str = Field(default_factory=lambda: new_id("vcframe"))
    batch_id: str
    video_id: str
    frame_index: int = Field(ge=0)
    timestamp_ms: int = Field(ge=0)
    image_path: str
    image_url: str = ""
    width: int | None = Field(default=None, ge=1)
    height: int | None = Field(default=None, ge=1)
    description_text: str = ""
    model_id: str = "deepseek-ai/deepseek-vl2-tiny"
    status: VideoCaptionStatus = "pending"
    error: str = ""
    result_json: dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=now_utc)
    updated_at: datetime = Field(default_factory=now_utc)

```

backend/app/repositories/__init__.py

```

"""Persistence adapters."""

```

backend/app/repositories/postgres_store.py

```

from datetime import datetime

from sqlalchemy import JSON, Boolean, DateTime, Float, Integer, String, Text, UniqueConstraint, create_engine,
func, or_, select, text
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column, sessionmaker

from app.models.schemas import (
    AnnotationsDocument,
    AuthUser,
    Asset,
    DetectTask,
    ImageAsset,
    ImageTask,
    ImageTaskResult,
    ImageTaskResultVersion,
    ImageTaskStatus,
    ImageTaskType,
    LabelConfig,
    VideoCaptionBatch,
    VideoCaptionFrame,
    VideoCaptionVideo,
    now_utc,
)

class Base(DeclarativeBase):
    pass

class AssetRecord(Base):
    __tablename__ = "assets"

```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
payload: Mapped[dict] = mapped_column(JSON)
```

```
class TaskRecord(Base):
    __tablename__ = "detect_tasks"
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
asset_id: Mapped[str] = mapped_column(String(64), index=True)
status: Mapped[str] = mapped_column(String(32), index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
payload: Mapped[dict] = mapped_column(JSON)
```

```
class AnnotationRecord(Base):
    __tablename__ = "annotations"
```

```
asset_id: Mapped[str] = mapped_column(String(64), primary_key=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
payload: Mapped[dict] = mapped_column(JSON)
```

```
class UserRecord(Base):
    __tablename__ = "app_user"
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
username: Mapped[str] = mapped_column(String(50), unique=True, index=True)
display_name: Mapped[str] = mapped_column(String(80))
password_hash: Mapped[str] = mapped_column(String(255))
role: Mapped[str] = mapped_column(String(32), default="student", index=True)
is_active: Mapped[bool] = mapped_column(Boolean, default=True, index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
```

```
class AuthSessionRecord(Base):
    __tablename__ = "auth_session"
```

```
token_hash: Mapped[str] = mapped_column(String(64), primary_key=True)
user_id: Mapped[str] = mapped_column(String(64), index=True)
expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
last_seen_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
```

```
class ImageAssetRecord(Base):
    __tablename__ = "image_asset"
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
original_name: Mapped[str] = mapped_column(String(255))
file_name: Mapped[str] = mapped_column(String(255))
file_url: Mapped[str] = mapped_column(String(500))
file_path: Mapped[str] = mapped_column(String(500))
mime_type: Mapped[str | None] = mapped_column(String(100), nullable=True)
file_size: Mapped[int] = mapped_column(Integer)
width: Mapped[int | None] = mapped_column(Integer, nullable=True)
height: Mapped[int | None] = mapped_column(Integer, nullable=True)
user_id: Mapped[str | None] = mapped_column(String(100), nullable=True, index=True)
session_id: Mapped[str] = mapped_column(String(100), default="default", index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
```

```
class ImageTaskRecord(Base):
    __tablename__ = "image_task"
    __table_args__ = (
        UniqueConstraint("image_id", "task_type", "user_key", "session_id", name="uk_image_task_scope"),
    )
```

```
id: Mapped[str] = mapped_column(String(64), primary_key=True)
```

```

image_id: Mapped[str] = mapped_column(String(64), index=True)
task_type: Mapped[str] = mapped_column(String(50), index=True)
status: Mapped[str] = mapped_column(String(50), default="pending", index=True)
user_id: Mapped[str | None] = mapped_column(String(100), nullable=True, index=True)
user_key: Mapped[str] = mapped_column(String(100), default="", index=True)
session_id: Mapped[str] = mapped_column(String(100), default="default", index=True)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))

```

```

class ImageTaskResultRecord(Base):
    __tablename__ = "image_task_result"

```

```

id: Mapped[str] = mapped_column(String(64), primary_key=True)
task_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
image_id: Mapped[str] = mapped_column(String(64), index=True)
task_type: Mapped[str] = mapped_column(String(50), index=True)
status: Mapped[str] = mapped_column(String(50), default="success", index=True)
result_image_url: Mapped[str] = mapped_column(String(500), default="")
result_image_path: Mapped[str] = mapped_column(String(500), default="")
result_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
annotation_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
description_text: Mapped[str] = mapped_column(Text, default="")
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
has_result: Mapped[bool] = mapped_column(Boolean, default=True)
review_status: Mapped[str] = mapped_column(String(32), default="draft", index=True)
submitted_by: Mapped[str | None] = mapped_column(String(64), nullable=True, index=True)
submitted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
reviewed_by: Mapped[str | None] = mapped_column(String(64), nullable=True, index=True)
reviewed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
review_comment: Mapped[str] = mapped_column(Text, default="")

```

```

class ImageTaskResultVersionRecord(Base):
    __tablename__ = "image_task_result_version"

```

```

id: Mapped[str] = mapped_column(String(64), primary_key=True)
result_id: Mapped[str] = mapped_column(String(64), index=True)
task_id: Mapped[str] = mapped_column(String(64), index=True)
image_id: Mapped[str] = mapped_column(String(64), index=True)
task_type: Mapped[str] = mapped_column(String(50), index=True)
version_no: Mapped[int] = mapped_column(Integer, index=True)
source: Mapped[str] = mapped_column(String(32), default="edited", index=True)
model_id: Mapped[str] = mapped_column(String(100), default="")
result_image_url: Mapped[str] = mapped_column(String(500), default="")
result_image_path: Mapped[str] = mapped_column(String(500), default="")
result_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
annotation_json: Mapped[dict | None] = mapped_column(JSON, nullable=True)
description_text: Mapped[str] = mapped_column(Text, default="")
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```

class LabelConfigRecord(Base):
    __tablename__ = "label_config"

```

```

label_id: Mapped[int] = mapped_column(Integer, primary_key=True)
english_name: Mapped[str] = mapped_column(String(120), index=True)
chinese_name: Mapped[str] = mapped_column(String(120), index=True)
description: Mapped[str] = mapped_column(Text, default="")
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```

class VideoCaptionBatchRecord(Base):
    __tablename__ = "video_caption_batch"

```

```

id: Mapped[str] = mapped_column(String(64), primary_key=True)
name: Mapped[str] = mapped_column(String(160), unique=True, index=True)
source_dir: Mapped[str] = mapped_column(String(1000), default="")
output_dir: Mapped[str] = mapped_column(String(1000), default="")

```

```

model_id: Mapped[str] = mapped_column(String(160), default="")
prompt: Mapped[str] = mapped_column(Text, default="")
frame_interval_seconds: Mapped[float] = mapped_column(Float, default=1.0)
status: Mapped[str] = mapped_column(String(32), default="pending", index=True)
total_videos: Mapped[int] = mapped_column(Integer, default=0)
total_frames: Mapped[int] = mapped_column(Integer, default=0)
metadata_json: Mapped[dict] = mapped_column(JSON, default=dict)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```
class VideoCaptionVideoRecord(Base):
```

```

    __tablename__ = "video_caption_video"
    __table_args__ = (
        UniqueConstraint("batch_id", "source_path", name="uk_video_caption_video_source"),
    )

```

```

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    batch_id: Mapped[str] = mapped_column(String(64), index=True)
    filename: Mapped[str] = mapped_column(String(255), index=True)
    display_name: Mapped[str] = mapped_column(String(500), default="")
    keywords_json: Mapped[list] = mapped_column(JSON, default=list)
    source_path: Mapped[str] = mapped_column(String(1000))
    frame_dir: Mapped[str] = mapped_column(String(1000), default="")
    fps: Mapped[float | None] = mapped_column(Float, nullable=True)
    width: Mapped[int | None] = mapped_column(Integer, nullable=True)
    height: Mapped[int | None] = mapped_column(Integer, nullable=True)
    frame_count: Mapped[int | None] = mapped_column(Integer, nullable=True)
    duration_ms: Mapped[int | None] = mapped_column(Integer, nullable=True)
    status: Mapped[str] = mapped_column(String(32), default="pending", index=True)
    error: Mapped[str] = mapped_column(Text, default="")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```
class VideoCaptionFrameRecord(Base):
```

```

    __tablename__ = "video_caption_frame"
    __table_args__ = (
        UniqueConstraint("video_id", "frame_index", name="uk_video_caption_frame_index"),
    )

```

```

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    batch_id: Mapped[str] = mapped_column(String(64), index=True)
    video_id: Mapped[str] = mapped_column(String(64), index=True)
    frame_index: Mapped[int] = mapped_column(Integer, index=True)
    timestamp_ms: Mapped[int] = mapped_column(Integer, index=True)
    image_path: Mapped[str] = mapped_column(String(1000))
    image_url: Mapped[str] = mapped_column(String(500), default="")
    width: Mapped[int | None] = mapped_column(Integer, nullable=True)
    height: Mapped[int | None] = mapped_column(Integer, nullable=True)
    description_text: Mapped[str] = mapped_column(Text, default="")
    model_id: Mapped[str] = mapped_column(String(160), default="")
    status: Mapped[str] = mapped_column(String(32), default="pending", index=True)
    error: Mapped[str] = mapped_column(Text, default="")
    result_json: Mapped[dict] = mapped_column(JSON, default=dict)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

```

```
class PostgresStore:
```

```

    def __init__(self, database_url: str) -> None:
        connect_args = {"check_same_thread": False} if database_url.startswith("sqlite") else {}
        self.engine = create_engine(database_url, pool_pre_ping=True, connect_args=connect_args)
        self.session_factory = sessionmaker(self.engine, expire_on_commit=False)
        Base.metadata.create_all(self.engine)
        self._ensure_label_config_schema(database_url)
        self._ensure_video_caption_schema(database_url)
        self._ensure_image_task_review_schema(database_url)
        self._ensure_admin_user()

```

```

    def create_asset(self, asset: Asset) -> Asset:

```

```
.panel-head {
  min-height: 48px;
  padding: 8px 10px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1px solid #e1e7ee;
}

.upload-zone {
  position: relative;
  min-height: 120px;
  margin: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  gap: 8px;
  border: 1px dashed #a8b7c6;
  border-radius: 8px;
  color: #405160;
  background: #f8fafc;
  cursor: pointer;
}

.task-mini-tabs {
  padding: 10px;
  align-items: stretch;
}

.task-mini-tabs button {
  flex: 1 1 45%;
  min-width: 0;
  font-size: 12px;
}

.upload-zone.compact {
  min-height: 72px;
}

.upload-zone input {
  position: absolute;
  inset: 0;
  opacity: 0;
  cursor: pointer;
}

.ghost-button {
  margin: 0 10px 10px;
  width: calc(100% - 20px);
}

.history-list {
  min-height: 0;
  padding: 0 10px 10px;
  display: grid;
  gap: 8px;
  overflow-y: auto;
}

.history-item {
  width: 100%;
  min-height: 72px;
  padding: 8px;
  display: grid;
  grid-template-columns: 58px minmax(0, 1fr);
  gap: 9px;
  align-items: center;
  justify-items: stretch;
  text-align: left;
  background: #f8fafc;
}
```

```
}

.history-item.selected {
  border-color: #286eac;
  background: #eef6ff;
}

.history-item img {
  width: 58px;
  height: 50px;
  object-fit: cover;
  border-radius: 6px;
  background: #e8edf2;
}

.history-item span {
  min-width: 0;
  display: grid;
  gap: 3px;
}

.history-item strong {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  font-size: 13px;
}

.editor-panel {
  min-width: 0;
  display: grid;
  grid-template-rows: auto auto minmax(0, 1fr) auto;
  overflow: hidden;
}

.editor-toolbar {
  padding: 10px 12px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  border-bottom: 1px solid #e1e7ee;
}

.editor-toolbar > div:first-child {
  display: grid;
  gap: 3px;
}

.annotation-toolbar {
  min-height: 46px;
  padding: 6px 12px;
  border-bottom: 1px solid #e1e7ee;
  background: #fbfcfe;
  flex-wrap: nowrap;
  overflow-x: auto;
}

.annotation-toolbar button {
  min-width: 36px;
  padding: 0 10px;
  white-space: nowrap;
}

.canvas-stage {
  position: relative;
  min-width: 0;
  min-height: 0;
  overflow: hidden;
  background:
    linear-gradient(45deg, #eef2f6 25%, transparent 25%),
```

```
    linear-gradient(-45deg, #eef2f6 25%, transparent 25%),
    linear-gradient(45deg, transparent 75%, #eef2f6 75%),
    linear-gradient(-45deg, transparent 75%, #eef2f6 75%);
background-position: 0 0, 0 10px, 10px -10px, -10px 0;
background-size: 20px 20px;
cursor: default;
}
```

```
.canvas-stage.tool-pan {
  cursor: grab;
}
```

```
.canvas-stage.tool-pan:active {
  cursor: grabbing;
}
```

```
.canvas-stage.tool-box,
.canvas-stage.tool-line,
.canvas-stage.tool-polygon {
  cursor: crosshair;
}
```

```
.processing-mask {
  position: absolute;
  inset: 0;
  z-index: 12;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 24px;
  background: rgba(15, 23, 42, 0.56);
  backdrop-filter: blur(2px);
}
```

```
.processing-card {
  width: min(360px, 100%);
  min-height: 180px;
  padding: 28px 24px;
  display: grid;
  justify-items: center;
  align-content: center;
  gap: 12px;
  border: 1px solid rgba(255, 255, 255, 0.22);
  border-radius: 8px;
  color: #ffffff;
  text-align: center;
  background: rgba(23, 32, 42, 0.92);
  box-shadow: 0 18px 45px rgba(15, 23, 42, 0.35);
}
```

```
.processing-card strong {
  font-size: 18px;
  line-height: 1.35;
}
```

```
.processing-card span {
  max-width: 100%;
  color: #d7dee6;
  font-size: 13px;
  overflow-wrap: anywhere;
}
```

```
.processing-spinner {
  color: #7dd3fc;
  animation: spin 900ms linear infinite;
}
```

```
.empty-canvas {
  position: absolute;
  inset: 0;
  display: flex;
}
```

```
align-items: center;
justify-content: center;
flex-direction: column;
gap: 12px;
color: #66717d;
}

.canvas-content {
position: absolute;
inset: 24px;
display: flex;
align-items: center;
justify-content: center;
transform-origin: center center;
transition: transform 120ms ease;
}

.base-image {
max-width: 100%;
max-height: 100%;
object-fit: contain;
user-select: none;
pointer-events: none;
}

.overlay-layer {
position: absolute;
max-width: 100%;
max-height: 100%;
width: auto;
height: auto;
inset: 0;
margin: auto;
overflow: visible;
pointer-events: none;
}

.overlay-layer.interactive {
pointer-events: auto;
}

.box-shape {
fill: rgba(225, 29, 72, 0.04);
stroke: #e11d48;
stroke-width: 4;
vector-effect: non-scaling-stroke;
}

.box-label {
fill: #e11d48;
font-size: 28px;
font-weight: 700;
paint-order: stroke;
stroke: #ffffff;
stroke-width: 4;
vector-effect: non-scaling-stroke;
pointer-events: none;
}

.segment-shape {
fill: rgba(40, 110, 172, 0.26);
stroke: #286eac;
stroke-width: 4;
vector-effect: non-scaling-stroke;
}

.line-shape {
stroke: #7c3aed;
stroke-width: 5;
stroke-linecap: round;
vector-effect: non-scaling-stroke;
}
```

```
}

.shape-group {
  cursor: pointer;
  pointer-events: visiblePainted;
}

.shape-group.selected .box-shape,
.shape-group.selected .segment-shape,
.shape-group.selected .line-shape {
  stroke: #f59e0b;
}

.resize-handle,
.draft-point {
  fill: #ffffff;
  stroke: #f59e0b;
  stroke-width: 4;
  cursor: nwse-resize;
  vector-effect: non-scaling-stroke;
}

.draft-shape {
  fill: rgba(245, 158, 11, 0.12);
  stroke: #f59e0b;
  stroke-width: 4;
  stroke-dasharray: 10 8;
  vector-effect: non-scaling-stroke;
}

.pose-bone {
  stroke: #0ea5e9;
  stroke-width: 5;
  stroke-linecap: round;
  vector-effect: non-scaling-stroke;
  pointer-events: none;
}

.keypoint {
  fill: #f59e0b;
  stroke: #ffffff;
  stroke-width: 3;
  vector-effect: non-scaling-stroke;
  pointer-events: none;
}

.view-controls {
  min-height: 48px;
  padding: 8px 12px;
  border-top: 1px solid #e1e7ee;
}

.thumbnail-card {
  margin: 10px;
  padding: 0;
  display: block;
  overflow: hidden;
  background: #f8fafc;
}

.thumbnail-card img {
  display: block;
  width: 100%;
  max-height: 160px;
  object-fit: contain;
}

.image-info {
  margin: 0;
  padding: 0 12px 12px;
  display: grid;
```

```
grid-template-columns: 70px minmax(0, 1fr);
gap: 8px;
font-size: 13px;
}

.image-info dt {
  color: #66717d;
}

.image-info dd {
  margin: 0;
  min-width: 0;
  overflow-wrap: anywhere;
}

.field-label {
  display: grid;
  gap: 6px;
  color: #405160;
  font-size: 13px;
}

.field-label input,
.field-label select,
.field-label textarea,
.label-row input,
.label-row textarea {
  width: 100%;
  min-width: 0;
  padding: 8px 10px;
  border: 1px solid #c9d2dc;
  border-radius: 6px;
  color: #17202a;
  background: #ffffff;
}

.field-label textarea,
.label-row textarea {
  resize: vertical;
}

.danger-button {
  width: 100%;
  margin-top: 10px;
  border-color: #fecdd3;
  color: #be123c;
  background: #fff1f2;
}

.result-summary {
  margin: 0 10px 10px;
  padding-top: 10px;
  border-top: 1px solid #e1e7ee;
}

.result-summary h2 {
  margin: 0 0 10px;
  font-size: 15px;
}

.review-heading,
.review-actions,
.user-admin-row,
.user-admin-row > span {
  display: flex;
  align-items: center;
}

.review-heading,
.user-admin-row {
  justify-content: space-between;
```

```
    gap: 10px;
}

.review-heading h2 {
    margin: 0;
}

.review-summary {
    display: grid;
    gap: 10px;
}

.review-status {
    padding: 3px 7px;
    border-radius: 4px;
    color: #405160;
    background: #e8edf2;
    font-size: 11px;
    font-weight: 700;
}

.review-status.status-pending_review {
    color: #7a5200;
    background: #fff1c7;
}

.review-status.status-approved {
    color: #17613b;
    background: #dff3e8;
}

.review-status.status-rejected {
    color: #9b2d27;
    background: #fbe4e2;
}

.review-comment {
    margin: 0;
    color: #405160;
    font-size: 12px;
    line-height: 1.5;
}

.review-summary textarea {
    width: 100%;
    resize: vertical;
}

.review-actions {
    gap: 8px;
}

.review-actions button {
    flex: 1;
}

.user-admin-header {
    margin-top: 24px;
}

.user-admin-list {
    border: 1px solid #d7dee6;
    border-radius: 8px;
    background: #ffffff;
}

.user-admin-row {
    min-height: 58px;
    padding: 9px 12px;
    border-bottom: 1px solid #e1e7ee;
}
```

```
.user-admin-row:last-child {
  border-bottom: 0;
}

.user-admin-row > span {
  min-width: 0;
  flex: 1;
  align-items: flex-start;
  flex-direction: column;
}

.user-admin-row small {
  color: #66717d;
}

.user-admin-row select {
  min-width: 120px;
  height: 36px;
}

.preset-heading {
  margin-bottom: 8px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
}

.preset-heading h2 {
  margin: 0;
}

.preset-heading > span {
  color: #66717d;
  font-size: 12px;
}

.label-preset-grid {
  max-height: 260px;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 6px;
  overflow-y: auto;
}

.preset-label-button {
  width: 100%;
  min-width: 0;
  min-height: 48px;
  padding: 5px 6px;
  display: grid;
  grid-template-columns: 30px minmax(0, 1fr);
  gap: 6px;
  justify-content: stretch;
  text-align: left;
  background: #f8fafc;
}

.preset-label-button kbd {
  width: 30px;
  min-height: 24px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  border: 1px solid #bdc8d4;
  border-bottom-width: 2px;
  border-radius: 4px;
  color: #405160;
  background: #ffffff;
  font-family: inherit;
```

```
font-size: 11px;
font-weight: 700;
}

.preset-label-button > span {
  min-width: 0;
  display: grid;
  align-content: center;
  gap: 2px;
}

.preset-label-button strong,
.preset-label-button small {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.preset-label-button strong {
  font-size: 12px;
}

.preset-label-button small {
  color: #66717d;
  font-size: 10px;
}

.preset-label-button.active kbd {
  border-color: #286eac;
  color: #145995;
  background: #ffffff;
}

.result-row {
  min-height: 36px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  border-bottom: 1px solid #edf1f5;
}

.result-row strong,
.result-row span {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.annotated-summary {
  max-height: 180px;
  overflow-y: auto;
}

.annotation-row {
  width: 100%;
  min-height: 34px;
  padding: 6px 8px;
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 8px;
  justify-items: stretch;
  text-align: left;
  background: #f8fafc;
}

.annotation-row.selected {
  border-color: #f59e0b;
  background: #fff7df;
}
```

```
}

.annotation-row strong,
.annotation-row span {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.annotation-row span {
  color: #66717d;
  font-size: 12px;
}

.caption-actions {
  margin-bottom: 8px;
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
}

.caption-actions button {
  flex: 1 1 116px;
  min-width: 0;
  min-height: 32px;
  padding: 0 8px;
  white-space: nowrap;
}

.caption-text {
  margin: 0;
  line-height: 1.6;
  color: #405160;
}

.history-viewing {
  margin-bottom: 8px;
  padding: 8px;
  border: 1px solid #f5c56b;
  border-radius: 6px;
  color: #7a4b00;
  background: #fff7df;
  font-size: 12px;
}

.version-summary {
  padding-top: 10px;
  max-height: 170px;
  overflow-y: auto;
}

.version-list {
  display: grid;
  gap: 4px;
}

.version-row {
  width: 100%;
  min-height: 34px;
  padding: 6px 8px;
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 8px;
  justify-items: stretch;
  text-align: left;
  background: #f8fafc;
}

.version-row.selected {
  border-color: #286eac;
```

```
    background: #eef6ff;
}

.version-row span {
  min-width: 0;
  display: flex;
  align-items: center;
  justify-content: flex-start;
  gap: 8px;
  overflow: hidden;
}

.version-row small {
  color: #66717d;
  font-size: 12px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.restore-button {
  margin: 10px 0 0;
  width: 100%;
}

.settings-page {
  flex: 1;
  min-height: 0;
  padding: 18px;
  overflow: auto;
}

.gesture-page {
  flex: 1;
  min-height: 0;
  padding: 16px;
  display: flex;
  flex-direction: column;
  gap: 14px;
  overflow: auto;
  background: #edf1f4;
}

.gesture-toolbar {
  min-height: 66px;
  padding: 10px 14px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  border: 1px solid #d3dce4;
  border-radius: 8px;
  background: #ffffff;
}

.gesture-toolbar h2 {
  margin: 0 0 4px;
  font-size: 18px;
  letter-spacing: 0;
}

.gesture-toolbar p {
  margin: 0;
  color: #66717d;
  font-size: 13px;
}

.gesture-actions,
.camera-state,
.result-panel-heading > span,
.privacy-note {
```

```
display: flex;
align-items: center;
gap: 8px;
}

.camera-state {
  min-width: 122px;
  color: #52606d;
  font-size: 13px;
}

.camera-state i {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: #8895a2;
}

.camera-state.state-running i {
  background: #1f9d67;
  box-shadow: 0 0 0 4px rgba(31, 157, 103, 0.13);
}

.camera-state.state-loading i {
  background: #d28b16;
}

.camera-state.state-error i {
  background: #c33d3d;
}

.gesture-layout {
  flex: 1;
  min-height: 440px;
  display: grid;
  grid-template-columns: minmax(0, 1fr) 300px;
  gap: 14px;
}

.camera-stage {
  position: relative;
  min-height: 0;
  overflow: hidden;
  border: 1px solid #25343e;
  border-radius: 8px;
  background: #101820;
}

.camera-stage video,
.camera-stage canvas {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: contain;
  transform: scaleX(-1);
}

.camera-stage canvas {
  pointer-events: none;
}

.camera-placeholder {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  gap: 10px;
  color: #dce6ec;
```

```
    text-align: center;
}

.camera-placeholder span {
  max-width: 460px;
  color: #9fb0bc;
  font-size: 13px;
}

.camera-metrics {
  position: absolute;
  top: 12px;
  left: 12px;
  display: flex;
  gap: 6px;
  pointer-events: none;
}

.camera-metrics span {
  padding: 5px 8px;
  border: 1px solid rgba(255, 255, 255, 0.25);
  border-radius: 4px;
  color: #ffffff;
  background: rgba(10, 20, 27, 0.72);
  font: 600 11px ui-monospace, SFMono-Regular, Menlo, monospace;
}

.gesture-results {
  min-height: 0;
  padding: 14px;
  display: flex;
  flex-direction: column;
  gap: 12px;
  overflow-y: auto;
  border: 1px solid #d3dce4;
  border-radius: 8px;
  background: #ffffff;
}

.result-panel-heading {
  padding-bottom: 12px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1px solid #e1e7ee;
}

.result-panel-heading span {
  font-weight: 700;
}

.result-panel-heading small {
  color: #7a8793;
}

.gesture-empty {
  min-height: 190px;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  gap: 8px;
  color: #7b8994;
  text-align: center;
}

.gesture-empty span {
  font-size: 12px;
}

.hand-result {
```

```
padding: 12px;
border: 1px solid #d8e1e8;
border-left: 4px solid #1f9d67;
border-radius: 6px;
background: #f9fbfc;
}

.hand-result header {
margin-bottom: 10px;
display: flex;
align-items: center;
justify-content: space-between;
gap: 8px;
}

.hand-result header span {
color: #62717d;
font-size: 12px;
}

.hand-result header strong {
color: #116645;
font-size: 16px;
}

.hand-result dl {
margin: 0;
display: grid;
grid-template-columns: 1fr auto;
gap: 6px 12px;
font-size: 13px;
}

.hand-result dt {
color: #66717d;
}

.hand-result dd {
margin: 0;
font-weight: 600;
}

.privacy-note,
.https-warning {
padding: 10px;
border-radius: 6px;
font-size: 12px;
line-height: 1.5;
}

.privacy-note {
margin-top: auto;
color: #275d49;
background: #edf8f3;
}

.https-warning {
color: #8a4d0f;
border: 1px solid #ecd2a7;
background: #fff8e9;
}

.retry-button {
width: 100%;
}

.video-caption-page {
flex: 1;
min-height: 0;
padding: 12px;
display: grid;
```

```

    grid-template-columns: 320px minmax(420px, 1fr) 380px;
    gap: 12px;
    overflow: hidden;
}

.video-list-panel,
.frame-browser,
.frame-detail-panel {
    min-height: 0;
    border: 1px solid #d7dee6;
    border-radius: 8px;
    background: #ffffff;
    overflow: hidden;
}

.video-list-panel,
.frame-detail-panel {
    display: flex;
    flex-direction: column;
}

.video-list-panel {
    display: grid;
    grid-template-rows: auto auto minmax(120px, 0.34fr) auto minmax(260px, 0.66fr);
}

.batch-summary {
    margin: 10px;
    padding: 10px;
    display: grid;
    gap: 5px;
    border: 1px solid #d7dee6;
    border-radius: 6px;
    background: #f8fafc;
}

.batch-summary span,
.video-row span,
.frame-toolbar span,
.frame-card span {
    color: #66717d;
    font-size: 12px;
}

.sub-head {
    border-top: 1px solid #e1e7ee;
}

.video-list {
    min-height: 0;
    padding: 0 10px 10px;
    display: grid;
    align-content: start;
    gap: 8px;
    overflow-y: auto;
}

.video-row {
    width: 100%;
    min-height: 82px;
    padding: 8px 10px;
    display: grid;
    justify-items: stretch;
    gap: 4px;
    text-align: left;
    background: #f8fafc;
}

.video-row.selected {
    border-color: #286eac;
    background: #eef6ff;
}

```

```
}

.video-row strong {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  font-size: 13px;
}

.batch-row {
  min-height: 52px;
}

.video-original-name {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.keyword-list {
  display: flex;
  align-items: center;
  gap: 5px;
  flex-wrap: wrap;
}

.keyword-list small {
  padding: 2px 6px;
  border-radius: 999px;
  color: #145995;
  background: #eef6ff;
  font-size: 11px;
}

.frame-browser {
  display: grid;
  grid-template-rows: auto minmax(0, 1fr);
}

.frame-toolbar {
  min-height: 58px;
  padding: 10px 12px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  border-bottom: 1px solid #e1e7ee;
}

.frame-toolbar > div {
  min-width: 0;
  display: grid;
  gap: 3px;
}

.frame-toolbar strong {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.frame-search {
  min-width: min(360px, 48%);
  display: grid;
  grid-template-columns: auto minmax(0, 1fr) auto;
  align-items: center;
  gap: 8px;
}
```

```
.frame-search input {
  width: 100%;
  min-width: 0;
  height: 36px;
  padding: 0 10px;
  border: 1px solid #c9d2dc;
  border-radius: 6px;
}

.frame-grid {
  min-height: 0;
  padding: 12px;
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(144px, 1fr));
  align-content: start;
  gap: 10px;
  overflow-y: auto;
}

.frame-card {
  min-width: 0;
  padding: 6px;
  display: grid;
  gap: 6px;
  background: #f8fafc;
}

.frame-card.selected {
  border-color: #286eac;
  background: #eef6ff;
}

.frame-card img {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  border-radius: 4px;
  background: #e8edf2;
}

.frame-preview {
  margin: 10px;
  min-height: 220px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid #e1e7ee;
  border-radius: 6px;
  background: #f8fafc;
  overflow: hidden;
}

.frame-preview img {
  display: block;
  width: 100%;
  max-height: 360px;
  object-fit: contain;
}

.detail-empty {
  margin: auto;
}

.settings-header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 16px;
}
```

```
.settings-header h2 {
  margin: 0;
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 22px;
}

.settings-header p {
  margin: 6px 0 0;
  color: #66717d;
}

.label-form {
  display: grid;
  grid-template-columns: minmax(160px, 1fr) minmax(160px, 1fr) minmax(240px, 1.6fr) auto;
  gap: 12px;
  align-items: end;
  padding: 14px;
  border: 1px solid #d7dee6;
  border-radius: 8px;
  background: #ffffff;
}

.label-table {
  margin-top: 14px;
  border: 1px solid #d7dee6;
  border-radius: 8px;
  overflow: hidden;
  background: #ffffff;
}

.label-table-head,
.label-row {
  display: grid;
  grid-template-columns: 72px 1fr 1fr 1.5fr 170px 170px 132px;
  gap: 10px;
  align-items: center;
  padding: 10px 12px;
  border-bottom: 1px solid #edf1f5;
  font-size: 13px;
}

.label-table-head {
  color: #405160;
  font-weight: 700;
  background: #f8fafc;
}

.label-row > span {
  min-width: 0;
  overflow-wrap: anywhere;
}

.row-actions {
  display: flex;
  align-items: center;
  gap: 6px;
}

.row-actions button {
  min-width: 32px;
  width: 32px;
  min-height: 32px;
  padding: 0;
}

.table-empty {
  padding: 24px;
}
```

```
.modal-backdrop {
  position: fixed;
  inset: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  justify-content: center;
  background: rgba(15, 23, 42, 0.62);
}

.system-loading-backdrop,
.system-error-backdrop {
  position: fixed;
  inset: 0;
  z-index: 40;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
  background: rgba(15, 23, 42, 0.5);
}

.system-loading-dialog,
.system-error-dialog {
  width: min(420px, 100%);
  padding: 20px;
  border: 1px solid #c9d2dc;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 18px 48px rgba(15, 23, 42, 0.24);
}

.system-loading-dialog {
  display: grid;
  justify-items: center;
  gap: 10px;
  text-align: center;
}

.system-loading-dialog svg {
  color: #1769aa;
}

.system-loading-dialog span,
.system-error-dialog span {
  color: #66717d;
  font-size: 13px;
}

.system-error-dialog {
  display: grid;
  gap: 18px;
  border-top: 4px solid #c53b32;
}

.system-error-dialog > div {
  display: grid;
  gap: 8px;
}

.system-error-dialog button {
  justify-self: end;
  min-width: 88px;
}

.auth-shell {
  min-height: 100vh;
  padding: 32px 20px;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```
    background: #eef1f4;
}

.auth-panel {
  width: min(420px, 100%);
  padding: 28px;
  border: 1px solid #cfd7df;
  border-top: 4px solid #1769aa;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 16px 42px rgba(23, 32, 42, 0.14);
}

.auth-brand {
  display: flex;
  align-items: center;
  gap: 12px;
}

.auth-brand img {
  width: 44px;
  height: 44px;
}

.auth-brand h1 {
  margin: 0 0 4px;
  font-size: 20px;
}

.auth-brand span {
  color: #66717d;
  font-size: 13px;
}

.auth-mode {
  margin: 24px 0 18px;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 6px;
}

.auth-form,
.auth-form label {
  display: grid;
  gap: 8px;
}

.auth-form {
  gap: 16px;
}

.auth-form label > span {
  color: #405160;
  font-size: 13px;
  font-weight: 600;
}

.auth-form input {
  width: 100%;
  min-width: 0;
  height: 42px;
  padding: 0 12px;
  border: 1px solid #bdc8d4;
  border-radius: 6px;
  color: #17202a;
  background: #ffffff;
}

.auth-form input:focus {
  border-color: #1769aa;
  outline: 2px solid rgba(23, 105, 170, 0.14);
}
```

```
}

.auth-submit {
  min-height: 42px;
  margin-top: 4px;
  border-color: #1769aa;
  color: #ffffff;
  background: #1769aa;
}

.original-modal {
  max-width: min(90vw, 1200px);
  max-height: 90vh;
  padding: 12px;
  display: grid;
  gap: 10px;
  border-radius: 8px;
  background: #ffffff;
}

.original-modal img {
  max-width: 100%;
  max-height: 76vh;
  object-fit: contain;
}

.spin {
  animation: spin 900ms linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

@media (max-width: 980px) {
  .gesture-page {
    padding: 10px;
  }

  .gesture-toolbar {
    align-items: stretch;
    flex-direction: column;
  }

  .gesture-actions {
    justify-content: space-between;
    flex-wrap: wrap;
  }

  .gesture-layout {
    grid-template-columns: 1fr;
    grid-template-rows: minmax(340px, 60vh) auto;
  }

  .gesture-results {
    overflow: visible;
  }

  .topbar {
    align-items: flex-start;
    flex-direction: column;
  }

  .topbar-actions {
    width: 100%;
    align-items: flex-start;
    flex-wrap: wrap;
  }
}
```

```

.user-session {
  min-width: min(220px, 100%);
  padding-left: 0;
  border-left: 0;
}

.image-task-workspace {
  height: auto;
  min-height: 100vh;
  overflow: visible;
}

.workspace-body,
.workspace-body[style] {
  grid-template-columns: 1fr;
  grid-template-rows: minmax(220px, auto) minmax(520px, 1fr) minmax(240px, auto);
  min-height: calc(100vh - 121px);
  overflow-y: auto;
}

.panel-resizer {
  display: none;
}

.history-panel,
.thumbnail-panel {
  min-height: 220px;
}

.history-panel.collapsed,
.thumbnail-panel.collapsed {
  width: auto;
  min-height: 48px;
}

.label-form,
.label-table-head,
.label-row {
  grid-template-columns: 1fr;
}

.label-table-head {
  display: none;
}
}

```

frontend/tsconfig.json

```

{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["DOM", "DOM.Iterable", "ES2022"],
    "allowJs": false,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "resolveJsonModule": true,
    "types": ["vite/client"],
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "preserve"
  },
  "include": ["src"],
}

```

```
"references": [{ "path": "./tsconfig.node.json" }]  
}
```

frontend/tsconfig.node.json

```
{  
  "compilerOptions": {  
    "composite": true,  
    "skipLibCheck": true,  
    "module": "ESNext",  
    "moduleResolution": "Bundler",  
    "allowSyntheticDefaultImports": true  
  },  
  "include": ["vite.config.ts"]  
}
```

frontend/vite.config.d.ts

```
declare const _default: import("vite").UserConfigFnObject;  
export default _default;
```

frontend/vite.config.js

```
import { defineConfig, loadEnv } from "vite";  
import vue from "@vitejs/plugin-vue";  
export default defineConfig(function (_a) {  
  var mode = _a.mode;  
  var backendTarget = loadEnv(mode, ".", "").VITE_PROXY_TARGET || "***"  
  return {  
    plugins: [vue()],  
    server: {  
      port: 4004,  
      proxy: {  
        "/api": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
        "/uploads": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
      },  
    },  
  };  
});
```

frontend/vite.config.ts

```
import { defineConfig, loadEnv } from "vite";  
import vue from "@vitejs/plugin-vue";  
  
export default defineConfig(({ mode }) => {  
  const backendTarget = loadEnv(mode, ".", "").VITE_PROXY_TARGET || "***"  
  return {  
    plugins: [vue()],  
    server: {  
      port: 4004,  
      proxy: {  
        "/api": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
        "/uploads": {  
          target: backendTarget,  
          changeOrigin: false,  
        },  
      },  
    },  
  };  
});
```

```
    },  
  },  
},  
};  
});
```

scripts/dev-backend.sh

```
#!/usr/bin/env bash  
set -euo pipefail  
  
cd "$(dirname "$0")/../../backend"  
uv sync --dev  
DATA_DIR="${DATA_DIR:-./data}" \  
INFERENCE_BACKEND="${INFERENCE_BACKEND:-ultralytics}" \  
MODEL_PATH="${MODEL_PATH:-./yolo11n.pt}" \  
DEVICE="${DEVICE:-0}" \  
RUN_TASKS_INLINE="${RUN_TASKS_INLINE:-true}" \  
TASK_WORKER_ENABLED="${TASK_WORKER_ENABLED:-false}" \  
uv run uvicorn app.main:app --reload --host *** --port "${PORT:-8010}"
```

scripts/dev-frontend.sh

```
#!/usr/bin/env bash  
set -euo pipefail  
  
cd "$(dirname "$0")/../../frontend"  
npm install  
VITE_API_BASE_URL="${VITE_API_BASE_URL:-*** \  
npm run dev -- --host *** --port "${PORT:-4004}"
```

scripts/smoke.sh

```
#!/usr/bin/env bash  
set -euo pipefail  
  
cd "$(dirname "$0")/../../backend"  
uv sync --dev  
DATA_DIR="$(mktemp -d)" INFERENCE_BACKEND=mock RUN_TASKS_INLINE=true uv run pytest
```

docker-compose.yml

```
services:  
  backend:  
    image: python:3.11-slim  
    working_dir: /app/backend  
    command: >  
    sh -c "pip install -e . &&  
    uvicorn app.main:app --host *** --port 8000"  
    environment:  
      DATA_DIR: /app/data  
      DATABASE_URL: ${DATABASE_URL}  
      INFERENCE_BACKEND: mock  
      TASK_WORKER_ENABLED: "true"  
      CORS_ORIGINS: ***  
    ports:  
      - "8010:8000"  
    volumes:  
      - ./app  
  
  frontend:  
    image: node:24  
    working_dir: /app/frontend  
    command: >  
    sh -c "npm install &&
```

```
npm run dev -- --host *** --port 4004"
environment:
  VITE_API_BASE_URL: ""
  VITE_PROXY_TARGET: ***
ports:
  - "4004:4004"
volumes:
  - ./app
depends_on:
  - backend
```

games/competition-positioning.md

2026 工业互联网技术与应用挑战赛参赛定位

赛事识别

赛事页面：<***

公开索引显示该赛事为“2026年第二届工业互联网技术与应用挑战赛——数智应用创新赛道”，主办方为中国互联网协会智慧教育工作委员会。移动端通知页显示有“关于举办2026年第二届工业互联网技术与应用挑战赛的通知.pdf”官方盖章文件。

第三方赛事索引显示报名/赛程区间为 2026.03.17-2026.07.24。当前日期为 2026-07-20，按该信息推算剩余时间很短，应优先准备可演示系统、作品说明和应用价值材料。

推荐赛道

推荐选择：数智应用创新赛道。

项目理由：

- 项目核心是“多模态铁路目标识别与图像/视频理解”，属于 AI + 工业场景应用。
- 已具备视频解帧、目标检测、DeepSeek-VL2 图像描述、数据库存储、前端检索展示等完整链路。
- 场景可落到铁路基础设施巡检、施工安全识别、设备状态记录、人员与风险识别，契合工业企业数智化转型。
- 面向东南亚铁路发展，可包装成跨区域、跨语言、低成本部署的铁路运维数智平台。

不建议的定位

不建议只叫“铁路多目标识别系统”。

原因：这个名字偏算法功能，比赛更看重工业互联网应用价值、场景闭环和推广性。应该把重点从“识别”提升到“巡检、运维、安全、国际化服务”。

推荐题目

首选题目：

面向东南亚铁路运维的多模态智能巡检与双语风险识别系统

英文题目：

Multimodal Intelligent Inspection and Bilingual Risk Recognition System for Southeast Asian Railway Operations

备选题目

1. 面向“一带一路”铁路基础设施的多模态智能巡检与安全预警平台
2. 面向东南亚铁路场景的视觉多模态设备识别与运维辅助系统
3. 基于视觉大模型的跨语言铁路巡检解译与安全风险识别平台
4. 面向国际铁路运维的多模态目标识别、语义解译与风险管理系统

作品定位一句话

本项目面向东南亚铁路建设与运维场景，构建集视频解帧、铁路目标检测、视觉语言描述、风险关键词提取、双语展示与数据管理于一体的多模态智能巡检平台，为跨语言铁路运维、施工安全监管和设备巡检提供低成本、可部署的数智化解决方案。

建议突出功能

- 视频批量解帧：无人机/移动巡检视频自动拆解为关键帧。
- 多模态识别：结合 YOLO 检测与 DeepSeek-VL2 图像描述。

- 铁路语义关键词：接触网、轨道、人员、设备、施工、安全风险等标签化展示。
- 数据库存储：批次、视频、帧、描述、关键词全量入库，便于追溯。
- 前端可视化：按视频浏览关键帧，查看语义描述与关键词。
- 国际化双语：中文/英文界面与报告，后续可扩展东南亚本地语言。

后续材料建议

- PPT 主线：痛点 -> 场景 -> 技术方案 -> 系统演示 -> 东南亚推广价值。
- 演示重点：打开视频解帧页面，展示 24 个 DJI 视频、1716 帧、关键词和描述。
- 数据亮点：真实无人机铁路视频、实际抽帧入库、可检索可追溯。
- 应用价值：降低巡检人工成本，提高跨语言协同效率，服务铁路出海和区域基础设施运维。

games/competition/analysis/评分对照与差距分析.md

评分对照与差距分析

一、结论

本项目符合“工业企业数智化转型综合解决方案”赛题，可选择官方列出的“设备智能运维与健康管理”方向。铁路基础设施属于典型工业装备与生产运营基础设施，当前系统已经形成视频采集、解帧、视觉语言描述、关键词抽取、PostgreSQL 入库和 Web 复核的真实闭环。

当前材料的主要问题不是方向不符，而是证据与文档结构没有完全围绕评分标准组织。按现有初稿保守估计约为 **65-72 分**。完成双语最小闭环、性能与价值量化、实施和投入产出章节后，材料才具备冲击 **85 分以上** 的基础。

二、官方任务要求对照

官方要求	当前符合情况	已有内容或证据	主要缺口
--- --- --- ---			
项目背景与痛点分析	基本符合	东南亚铁路场景、视频回看成本、多源数据和跨语言协同痛点	缺少明确的目标用户、真实作业流程、痛点基线和需求指标
总体方案设计	较好符合	分层架构、采集架构、识别流程、数据库 ER 图、部署架构	缺少系统边界图、完整业务流程、数据流向和外部系统接口
创新点说明	部分符合	多模态、术语归一化、多语种预留、视频语义命名	没有独立按技术/模式/应用/价值四类论证，也缺少竞品或传统方案对比
落地可行性	部分符合	FastAPI、Vue、PostgreSQL、本地 DeepSeek-VL2、边缘中心部署设想	缺少分阶段实施步骤、硬件清单、软件依赖、网络与安全设计、预算和风险控制
推广价值	较弱符合	提到跨区域部署和东南亚多语种扩展	缺少复制对象、标准化交付包、商业模式、收费对象和规模化路线

三、评分项与当前分值预估

评分项	满分	当前预估	判断依据	冲分动作
--- ---: ---: ---				
需求与场景契合度	15	12-13	场景真实、方向明确，有 24 段 DJI 铁路视频	增加角色、作业流程、业务边界和可量化需求目标
方案创新性	25	14-17	技术组合完整，但多语种目前主要是架构预留	做出中英双语最小闭环；增加四类创新和横向对比表
技术先进性与可行性	25	19-21	已有可运行前后端、数据库和本地模型	补部署规格、接口/数据流、实施计划、实测吞吐与失败恢复机制
应用价值与成效	20	8-11	只有 24 视频、1716 帧的处理规模，没有效果和成本指标	补效率、准确性、人工复核时间、成本节约和安全提升测算
文档完整性与规范性	15	11-12	图较多、逻辑清楚，但仍标“初稿”，参考文献不完整	改成方案说明书，增加目录、图表清单、证据附录和完整引用
合计	100	64-74	中位估计约 69	先补 45 分权重对应的创新与量化成效

分值是依据当前材料的内部预评，不代表组委会最终评分。

四、必须纠正的表述风险

1. “双语风险识别”目前主要是架构预留。正式稿应写成“已实现中英术语映射和双语展示”之前，必须先做出可操作界面、术语表和样例输出；否则改为“面向双语扩展的风险识别系统”。
2. 当前 1716 帧处理结果主要证明 DeepSeek-VL2 图像描述与入库链路。若没有同批次 YOLO 检测记录，不应把 YOLO 的效果与这批验证数据合并表述。
3. “降低成本、提高效率、提升安全”必须给出基线、测量口径、公式和样本量，不能只写定性结论。
4. 东南亚适配不能只依赖标题。至少需要覆盖热带环境、弱网络、跨语言协作、本地术语和边缘部署中的两个真实约束。
5. 参考文献目前存在机构报告或规范名称过于笼统的问题。正式稿应补完整题名、年份、发布机构和可核验来源。

五、最高优先级补强项

P0：提交前必须完成

- 将正式 PDF 从“学术论文初稿”改为“工业互联网数智应用解决方案说明书”。
- 增加四类创新点对照表：技术创新、模式创新、应用创新、价值创新。
- 增加实施路线、软硬件配置、部署拓扑、数据安全和故障恢复。
- 增加投入产出模型，至少提供保守/基准两种测算，并声明数据来源和假设。
- 使用真实数据补充处理耗时、单视频平均时长、帧处理吞吐、人工复核耗时和关键词抽检结果。
- 完成最小中英双语闭环：术语表、接口字段、前端切换或双语结果、人工审校状态。

P1：显著增分

- 增加传统人工巡检、单一 YOLO、通用视觉大模型和本方案的对比矩阵。
- 选取 3 个端到端案例，展示原视频、关键帧、中文描述、英文描述、风险级别和复核结果。
- 增加系统边界图、业务流程图、数据流图、边缘-中心部署图和价值闭环图。
- 补充可复制交付包与商业模式：软件许可、项目实施、模型/术语库定制、年度运维服务。

P2：有余力再做

- 增加泰语、老挝语或越南语中的一种术语样例，但不能用少量机器翻译样例冒充完整能力。
- 增加弱网缓存、断点续传或边缘推理演示。
- 对关键词抽取进行小规模标注评估，给出准确率、召回率和错误案例。

六、量化证据建议

指标	建议口径	当前状态
数据规模	视频数、总时长、抽帧数、场景类别数	已有视频数和抽帧数，缺总时长与类别分布
处理效率	总处理时间、帧/秒、单分钟视频处理时间、GPU 型号	缺失
描述质量	随机抽检 100 帧，人工评定正确/部分正确/错误	缺失
关键词质量	人工标注集上的 Precision、Recall、F1	缺失
双语质量	术语一致率、人工审校通过率、典型错误	未实现
人工节省	人工全量回看时间与系统筛查+复核时间对比	缺失
经济价值	年视频量、人工单价、算力与维护成本、回收期	缺失
安全价值	风险发现提前量、漏检下降或复核覆盖率	缺失

所有效果数字都应来自实测或明确标注为“测算值”，并附计算公式，避免将预测值写成既成成效。

games/research/ecrl-background.md

马东铁项目背景：事实依据与写作边界

更新日期：2026-07-21

本文档用于支撑参赛论文《面向东南亚铁路运维的多模态智能巡检与双语风险识别系统》的项目背景、答辩陈述和后续资料核验。优先采用马来西亚政府、中国政府及项目相关机构公开资料。

可直接引用的公开事实

主题	已核验信息	来源
项目定位	马东铁连接哥打巴鲁与巴生港；MRL持有项目与资产，CCCC承担工程、采购、施工和调试	[马来西亚交通部项目概况](***)
最新进度	线路全长665公里；截至2026年1月整体进度91.70%；哥打巴鲁至鹅唛段计划2026年底完成、2027年1月商业运营	[马来西亚交通部2026年2月新闻稿](***)
联合运维	MRL与CCCC按50:50股比成立运营维护合资公司	[“一带一路”网转新华社报道](***)
培训规模	运营维护阶段总体培养目标为3200人；2025年计划安排210名马来西亚学员赴柳州开展一年强化培训，首批102人启程	[马来西亚交通部PLKI-ECRL O&M讲话](***)
培训岗位	车站助理、信号技术员、机车副司机、接触网技术员、电力机车与动车组维护技术员	[马来西亚交通部PLKI-ECRL O&M讲话](***)
本地化目标	马方明确将培养本地能力、避免长期依赖外国专业力量作为人才计划目标	[马来西亚交通部PLKI-ECRL O&M讲话](***)
中马合作方向	双方将马东铁、铁路运输、职业教育、数字教育、人工智能和技术转移列为合作领域，并强调相互信任和互利	[马来西亚外交部中马联合声明](***)
东盟区域方向	中国与东盟行动计划提出职业技术教育交流、知识分享、技能发展、数字教育资源共建共享和语言交流	[中国外交部：中国—东盟行动计划（2026—2030）](***)

基于事实提出的方案判断

以下内容属于本项目对公开事实的工程化推导，不应表述为有关部门已经确认的既有事实：

1. 公开资料能够证明培训和联合运维正在推进，但不能据此断言马东铁“没有运维系统”。
2. 多岗位、跨语言和联合运维会产生培训资料、现场视频、双语术语、风险案例与复核记录的统一管理需求，因此存在建设数字化知识承载平台的合理空间。
3. “培训解决人会不会，平台解决知识能否持续沉淀”是本项目的价值主张，不是官方原文。
4. 对中马合作宜使用“技术认可、互利合作、能力共建、本地化运营”等表达，不使用“东盟依赖中国技术”作为事实结论。
5. 当前原型已实现中文多模态解帧、描述、关键词、数据库和前端浏览；中英双语、马来语与其他东南亚语种属于架构预留和后续建设内容。

答辩推荐表述

> 马东铁已进入从工程建设向运营准备转换的关键阶段。中马联合运维和大规模赴华培训正在建立本地专业队伍，但人员回国后仍需要一个能把培训知识、现场巡检视频、风险术语和处置记录持续沉淀下来的数字平台。本项目以真实铁路巡检视频为基础，先完成多模态识别、结构化入库和前端复核，再逐步扩展中英及马来语知识服务，服务的是技术转移后的本地能力固化，而不是替代专业人员。

games/submission/参赛方案目录.md

正式参赛方案建议目录

建议保留作品题目《面向东南亚铁路运维的多模态智能巡检与双语风险识别系统》，将文档副标题改为“工业互联网数智应用解决方案说明书”。正文建议控制在 22-30 页；赛题未规定页数，以证据完整、图表可读为准。

封面与摘要

- 封面：作品名称、赛道、方向、团队信息、日期。
- 执行摘要：用一页说明服务对象、核心痛点、方案闭环、已实现成果和量化价值。
- 目录、图目录和表目录。

1 项目背景与典型场景（对应需求契合度 15 分）

- 1.1 东南亚铁路建设与运维背景
- 1.2 目标客户与用户角色
- 1.3 现有巡检业务流程和核心痛点
- 1.4 数智化需求、建设范围和量化目标
- 1.5 典型应用场景：接触网、轨道、人员作业、设备间与环境风险

建议图表：现状业务流程图、用户角色图、痛点-需求-目标映射表。

2 总体解决方案

- 2.1 建设思路与设计原则
- 2.2 系统边界及外部参与方
- 2.3 总体技术架构
- 2.4 巡检业务闭环
- 2.5 视频与语义数据流程
- 2.6 功能模块和权限边界

建议图表：系统上下文图、总体架构图、业务流程图、数据流图。

3 核心技术与实现路径（对应技术先进性与可行性 25 分）

- 3.1 多源巡检数据接入与标准化
- 3.2 视频解帧、时序索引和关键帧组织
- 3.3 YOLO 目标检测及人工标注闭环
- 3.4 DeepSeek-VL2 场景描述
- 3.5 铁路关键词抽取与风险分级
- 3.6 中英双语术语库、生成与人工审校
- 3.7 PostgreSQL 数据模型与可追溯机制
- 3.8 Web 可视化和人工复核
- 3.9 边缘-中心协同部署、弱网缓存与安全设计

建议图表：模型协同图、风险识别流程、ER 图、部署拓扑、接口时序图。

4 方案创新性（对应创新性 25 分）

- 4.1 技术创新：检测、视觉语言和铁路知识融合

- 4.2 模式创新：机器初筛、双语解释、人工复核闭环
- 4.3 应用创新：面向跨语言铁路运维的语义化视频资产
- 4.4 价值创新：从“视频留档”升级为“风险知识服务”
- 4.5 与人工方案、单模型方案和通用平台的对比

本章必须用对比表和已实现证据支撑，不要只列口号。

5 系统功能与真实案例

- 5.1 已实现功能清单
- 5.2 24 段视频与 1716 帧数据概况
- 5.3 案例一：接触网与轨道巡检
- 5.4 案例二：施工人员与安全风险
- 5.5 案例三：设备间与环境风险
- 5.6 中英双语输出与人工审校案例

每个案例采用“输入-模型结果-风险解释-人工复核-处置建议”的一致结构。

6 系统验证与应用成效（对应应用价值与成效 20 分）

- 6.1 验证环境和评价方法
- 6.2 处理性能和稳定性
- 6.3 描述、关键词和双语质量
- 6.4 人工巡检效率对比
- 6.5 安全、质量和管理价值
- 6.6 投入产出与回收期测算
- 6.7 局限性和适用边界

建议图表：测试环境表、指标结果表、人工与系统对比图、成本收益瀑布图。所有数值注明“实测”或“测算”。

7 落地实施与运行保障

- 7.1 单站点部署方案
- 7.2 软件、硬件和网络配置清单
- 7.3 四阶段实施计划：调研、试点、验收、推广
- 7.4 数据安全、权限、日志与备份
- 7.5 模型更新、术语维护和人工审核机制
- 7.6 项目风险与应对措施

建议图表：实施甘特图、软硬件清单、运维闭环图、风险矩阵。

8 推广价值与商业模式

- 8.1 东南亚不同国家和线路的复制方法
- 8.2 从铁路向轨道交通、港口专用线和工业园区的扩展
- 8.3 标准化交付包与生态合作
- 8.4 软件许可、项目实施、定制与年度服务模式
- 8.5 三年推广路线

9 结论

只总结已验证成果、可量化价值和下一步落地，不重复背景。

附录

- A. 核心术语中英对照表。
- B. 接口与数据库字段摘要。
- C. 测试样本、计算公式和原始统计说明。
- D. 系统截图、演示访问说明或视频二维码。
- E. 原创性、数据授权和开源组件说明。
- F. 参考文献。

与当前论文目录的调整关系

当前章节	调整方式
--- ---	
引言、需求分析	合并并增强为第 1 章，补角色、流程和指标
系统总体架构	扩展为第 2 章，补系统边界、业务流和数据流
多模态识别方法	纳入第 3 章，并严格区分已实现与规划能力
多语种知识服务	纳入第 3.6 节，并用真实双语样例验证
数据管理与系统实现	分拆到第 3 章和第 5 章

系统验证与结果分析 | 重写为第 5、6 章，增加效果和价值指标 |
跨区域部署与应用前景 | 拆为第 7、8 章，补实施、预算和商业模式 |
结论与展望 | 删除泛化展望，改为已验证结论与近期落地计划 |